AI for Ecologists

A Primer

Savvas Paragkamian

September 17, 2026

Written with the assistance of AI agents.
Every claim, citation and recommendation was checked by the author,
who is responsible for all of it.

Contents

Preface **vii**

I The Landscape **1**

1 Chatbots, Models and Agents **2**

1.1 The model 2

1.1.1 What "training" produced 2

1.1.2 Emergence, and whether it is real 2

1.2 The chatbot 3

1.3 The agent and the harness 3

1.3.1 Why the harness matters more than the model 3

1.3.2 Files are all you need 4

1.4 The ecosystem, briefly 4

1.5 Where agents are going 4

2 Why Now? **6**

2.1 Compute 6

2.1.1 The memory wall 6

2.2 Architecture 7

2.3 Data, and who paid for it 7

2.4 Convergence, and what it means for a field like ours 7

2.5 Is this a bubble? 8

3 Powers and Limitations **9**

3.1 What language models are genuinely good at 9

3.2 Hallucination 9

3.2.1 The citation problem 10

3.3 Sycophancy 10

3.4 Dependence and cognitive offloading 11

3.5 Provenance 11

3.6 Jaggedness and the benchmark trap 11

3.7 Context windows and what falls out of them . . . 12
3.8 A note on AI-generated text detection . . . 12

4 Applications in Biology and Ecology 13
4.1 The pattern behind the successes . . . 13
4.2 Where ecology has genuine wins . . . 13
4.2.1 Image classification . . . 13
4.2.2 Natural history collections . . . 14
4.2.3 Text mining the literature . . . 14
4.2.4 Microbiome and multi-omics . . . 14
4.2.5 Curation support . . . 14
4.2.6 Query generation . . . 15
4.3 Where the claims outrun the evidence . . . 15
4.4 Agents in the life sciences . . . 16
4.5 Domain assistants . . . 16

II Data 17

5 AI-Ready Data 18
5.1 What the term conceals . . . 18
5.2 Standards worth knowing . . . 18
5.2.1 Data description . . . 18
5.2.2 Machine interfaces . . . 19
5.2.3 Data harmonisation in practice . . . 19
5.3 Infrastructure . . . 19
5.3.1 Biodiversity platforms . . . 19
5.3.2 Model and dataset platforms . . . 19
5.3.3 Standards and governance bodies . . . 20
5.4 Open weights, open source, and the difference . . . 20
5.5 Retrieval-augmented generation . . . 21
5.5.1 Where it breaks . . . 21
5.5.2 An alternative worth considering . . . 22
5.6 Provenance as a first-class requirement . . . 22

6 Organising and Protecting Your Data 23

6.1 Why this matters more now . . . 23
6.2 PARA . . . 23
6.3 Naming . . . 24
6.4 Raw data is read-only . . . 24
6.5 Backup: the 3-2-1 rule . . . 25
6.5.1 RAID is not a backup . . . 25
6.5.2 Verify restores . . . 25
6.6 Version control . . . 25
6.7 Plain text lasts . . . 26

III Working with Agents 27

7 The Agentic Coding Workflow 28
7.1 Terminology, briefly . . . 28
7.2 Architecture: chatbot versus harness . . . 28
7.3 The while loop . . . 29
7.4 A file is all you need . . . 29
7.4.1 README.md . . . 29
7.4.2 AGENTS.md . . . 29
7.4.3 Skills and packaged instructions . . . 30
7.4.4 Memory . . . 30
7.5 Tokens, context and cost . . . 30
7.6 Prompting, practically . . . 31
7.7 The review discipline . . . 31
7.8 Scope control . . . 31

8 The Software Stack 33
8.1 Layers . . . 33
8.2 Interfaces . . . 33
8.3 Ten rules for code that survives . . . 34
8.4 Local model tooling . . . 34
8.4.1 Runtimes . . . 34
8.4.2 Interfaces . . . 35
8.4.3 A working configuration . . . 35

8.5 Speech-to-text . . . 36
8.5.1 A failure mode worth knowing . . . 36

9 Reproducibility When the Tool Varies 37
9.1 The vocabulary . . . 37
9.2 Where a language model breaks this . . . 37
9.2.1 Sampling is stochastic . . . 37
9.2.2 The model changes underneath you . . . 38
9.2.3 There is no provenance for a generated claim . . . 38
9.2.4 Agents take different paths . . . 38
9.2.5 Context is invisible state . . . 38
9.3 A logging convention . . . 38
9.4 What to write in a methods section . . . 39
9.5 Containers, and their limits . . . 40

IV Local-First 41

10 The Local-First Approach 42
10.1 The argument . . . 42
10.1.1 Nothing leaves the machine . . . 42
10.1.2 No marginal cost . . . 42
10.1.3 It works offline . . . 42
10.1.4 Digital sovereignty . . . 42
10.1.5 Longevity . . . 43
10.2 The costs, stated honestly . . . 43
10.2.1 Capability . . . 43
10.2.2 Hardware and maintenance . . . 43
10.2.3 Speed . . . 43
10.3 Open, closed, and the layers between . . . 44
10.3.1 A sensible hybrid policy . . . 44
10.4 A reference architecture . . . 44
10.4.1 The logging proxy . . . 45
10.5 A FOSS stack, concretely . . . 45
10.6 Decentralised ecology . . . 46

11 Hardware for Local Inference **47**

11.1 Why memory, not compute . . . 47
11.2 Unified memory architecture . . . 47
11.3 Machines, and what they run . . . 47
11.3.1 Reading the table . . . 48
11.4 Sizing by task . . . 48
11.5 Quantisation . . . 48
11.6 Models worth having locally . . . 49
11.7 Shared deployment . . . 49

V Risk, Integrity and Keeping Up **51**

12 Security, Biorisk and Governance **52**

12.1 Running an agent is running arbitrary code . . . 52
12.2 Prompt injection . . . 52
12.3 Sandboxing . . . 53
12.4 Supply chain . . . 54
12.5 A practical baseline . . . 54
12.6 Dual use in biology . . . 54
12.6.1 The correlation with cybersecurity . . . 55
12.7 Governance and risk taxonomy . . . 55
12.7.1 What an institutional policy should cover . . . 56
12.8 The wider frame . . . 56

13 Changes Across the Scientific Workflow **57**

13.1 Literature retrieval . . . 57
13.1.1 Resources . . . 57
13.1.2 The verification requirement . . . 57
13.2 Reading . . . 58
13.3 Analysis . . . 58
13.4 Writing . . . 59
13.4.1 Policy . . . 59
13.5 Peer review and editorial work . . . 59
13.6 Synthesis and assessment . . . 60

13.7 Detection and its misuse . . . 60
13.8 A summary table . . . 60

14 How to Keep Up **61**
14.1 Why it is impossible, structurally . . . 61
14.2 What is actually stable . . . 61
14.3 People worth following . . . 61
14.4 Sources, by type . . . 62
14.4.1 Primary . . . 62
14.4.2 Curated lists . . . 62
14.4.3 Distribution and documentation . . . 62
14.4.4 Community . . . 62
14.5 A sustainable routine . . . 63
14.6 Learning by building . . . 63
14.7 What to tell colleagues who ask . . . 63
14.8 A closing note . . . 64

VI Appendices **65**

A Glossary **66**

B Checklists **69**
B.1 Before starting an AI-assisted project . . . 69
B.2 Before each agent session . . . 69
B.3 After each agent session . . . 69
B.4 Before citing anything . . . 70
B.5 Before submitting a manuscript . . . 70
B.6 Before reviewing a manuscript . . . 70
B.7 Evaluating a tool before adopting it (SNIFF) . . . 70
B.8 Sizing hardware for local inference . . . 71
B.9 Institutional AI policy: minimum contents . . . 71

References **72**

Preface

This is written for ecologists, taxonomists, biodiversity informaticians and the people who keep the databases running. It assumes you can write a script, that you have opinions about data quality, and that someone has recently asked you what you are doing about AI.

It does not assume you want to become a machine learning researcher. Almost nobody in ecology does. What most people need is a working idea of what these systems are, an honest account of what they can and cannot do, and enough practical detail to use them without damaging their data, their credibility or their institution's security.

Three things this book argues

The interesting change is not the chatbot. It is the loop that lets a model read files, run commands, look at the result and try again. Chat interfaces made language models visible; that loop made them consequential. The change in daily work is from writing code to specifying it, constraining what it can touch, and checking what came back. That is closer to running a system than to programming it.

Data work is the bottleneck, and ecology already did most of it. The phrase "AI-ready data" arrived late to a community that has spent thirty years building Darwin Core, GBIF, ontologies and curated databases by hand. Those resources are what makes any of this work in our field. It is worth being clear about who built them, on what funding, and on what terms they are now being used.

Running models on your own machine is a reasonable choice, not nostalgia. It costs more up front and less per query, keeps unpublished data off other people's servers, survives a service being withdrawn, and is the only option that works on a research vessel with no connection. It is not always right. It should always be on the table.

What this book is not

It is not a survey of model architectures and it will not teach you to fine-tune anything. It is not a table of benchmark scores; those go stale in weeks. It is not an argument that ecology should adopt AI, nor that it should refuse. The reading behind it includes a paper arguing forcefully that ecology is *not* ready (Mishra 2026) — that biased sampling, patchy data and missing causal structure make many current applications premature. That argument is taken seriously here rather than answered.

How to read it

Part I is the landscape: what these systems are, why they arrived when they did, what they do well and where they fail. Part II is data. Part III is working with agents day to day. Part IV is the case for running things locally, with prices. Part V covers security, dual-use risk, scientific integrity, and keeping up with a field that moves faster than the review cycle.

Chapters are short and mostly independent. Skip freely. If you read only one, read Chapter 3.

On dates and numbers

Model names, prices and benchmark figures here were accurate when written and will not stay that way. They are included because concrete examples teach better than abstractions, not because the particular values matter. Where a claim is about something slower-moving — memory bandwidth, the cost of inference, why models invent citations — it is flagged as such.

How this book was written

With AI agents, extensively: for drafting, for reading the source literature, and for assembling the bibliography. A book recommending a set of practices ought to have been produced under them, so it was.

Every entry in the bibliography corresponds to a document in the project's own literature directory. Nothing was cited from a model's memory — which is the failure Chapter 3 describes, and the reason Chapter 13 insists on resolving every identifier before it reaches a manuscript. That check was then run on this bibliography: all 39 DOIs resolve through Crossref and all 23 arXiv identifiers resolve through the arXiv API, with titles matching the entry in each case. The remaining references are reports, books and web pages that carry no such identifier and were verified by hand.

Doing this took about ten minutes of scripted work, which is the point.

The errors that remain are mine.

Acknowledgements

The reading behind this was assembled over two years of the Kybele project and ELIXIR-GR work at HCMR.

Part I

The Landscape

1. Chatbots, Models and Agents

Three words get used interchangeably and mean quite different things. Getting them apart is the first useful step.

1.1 The model

A large language model is a function. It takes a sequence of tokens and returns a probability distribution over the next token. That is the whole interface. Everything else — conversation, tool use, reasoning, refusal — is built on top of that one operation, repeated.

The model itself is stateless. It has no memory of your previous question, no file system, no clock, and no ability to act. Each call is independent. If a system appears to remember something, that is because the text was fed back in.

Key point. *The model is a calculator, not a colleague. It has no persistent state. Anything that looks like continuity is machinery around the model, not in it.*

1.1.1 What "training" produced

Pretraining fits the next-token function on a very large corpus. This is where the model acquires syntax, factual associations, code idioms, and the statistical shape of academic prose. A second stage — instruction tuning and preference optimisation — reshapes the outputs to be helpful, to follow instructions, and to decline certain requests.

That second stage matters more than it looks. It is where sycophancy comes from (Chapter 3), where refusal behaviour is installed, and where the model learns to guess confidently rather than say it does not know (Kalai et al. 2025).

1.1.2 Emergence, and whether it is real

Claims that language models display *emergent* capabilities — abilities that appear abruptly at scale rather than improving smoothly — have been central to the field's self-description. Krakauer et al. (2026) examine these claims from a complex systems perspective and are careful about them: much apparent emergence is an artefact of discontinuous evaluation metrics, and the deeper question of whether LLMs possess emergent *intelligence* rather than emergent *capability* remains open.

Mitchell (2026) names the practical version of this problem: *jagged intelligence*. Performance is not a single number. A system that handles a research-level reasoning task can fail at counting, at a spatial relation, or at a variant of the same question with different surface form. The jaggedness is not noise around a trend; it is the shape of the capability itself. This is why benchmark scores translate so poorly into expectations about your own task.

There is also a formal strand arguing that the architecture is more legible than its reputation suggests. Coppola (2026) proves that a sigmoid transformer implements weighted loopy belief propagation on an implicit factor graph — one layer is one round of belief propagation.

Whether this reframing changes anything in practice is unsettled, but it does undercut the claim that transformers cannot be understood.

1.2 The chatbot

A chatbot is a model plus a conversation buffer plus a web interface. The buffer is the important part: it concatenates the history and resubmits it on every turn. This is why long conversations get expensive, why they eventually forget the beginning, and why starting fresh often fixes a stuck session.

Chatbots are good for: drafting, explaining, translating, rephrasing, brainstorming, reading a paper you would otherwise skim. They are bad for anything requiring verified fact, current information, or action in the world.

1.3 The agent and the harness

An *agent* is a model given tools and a loop. An *agent harness* is the orchestration system that manages that loop: it decides what the model sees, executes the tool calls the model requests, feeds results back, enforces permissions, and decides when to stop.

The core is unremarkable. Liu et al. (2026) analysed the architecture of a production coding agent and found the centre of the system to be a simple while-loop: call the model, run the tools it asks for, repeat. Most of the code is not in the loop. It is in the systems around it — the permission layer, the context manager, the memory, the error recovery.

Listing 1.1: The whole idea, in pseudocode

```
while not done:
    response = model(context)
    if response.wants_tool:
        result = run_tool(response.tool, response.args)   # <- permissions here
        context += result
    else:
        done = True
```

That single `run_tool` line is where the security risk lives. It is the point at which a text generator becomes something that can delete your files.

1.3.1 Why the harness matters more than the model

Two harnesses running the same model behave very differently. The differences come from:

- **Context assembly** — what goes into the prompt, in what order, and what gets dropped when space runs out.
- **Tool design** — a good file-edit tool that fails loudly beats a clever one that fails silently.
- **Permission model** — what runs without asking, what prompts, what is refused outright.

- **Memory** — what persists across sessions, and in what form.

Du (2026) surveys agent memory as a *write–manage–read* loop coupled to perception and action, spanning context-resident compression, retrieval-augmented stores, reflective self-improvement and hierarchical virtual context. The taxonomy is useful, but the practical finding for our purposes is simpler: the memory that works most reliably today is a plain file on disk that the agent reads at the start of every session.

1.3.2 Files are all you need

Piskala (2026) traces this explicitly to Unix. The 1970s abstraction — *everything is a file* (Ritchie and Thompson 1974) — collapsed heterogeneous resources into one uniform interface, and it survived fifty years of hardware change. Agents face the analogous problem: databases, APIs, cloud infrastructure, memories, human input, each with its own protocol. The same solution works. Designing an agent around file-like abstractions makes it inspectable, versionable, greppable and diffable.

This has a direct consequence for how you set up a project. A `README.md` that explains the repository and an `AGENTS.md` (or `CLAUDE.md`) that states the conventions will do more for agent performance than most prompt engineering. They are plain text, they live in version control, and you can read them yourself.

Key point. *`AGENTS.md` is not a wish list. It is a behavioural contract that closes specific failure modes you have actually observed. Write it after something goes wrong, not before.*

1.4 The ecosystem, briefly

The tooling layer is crowded and turns over fast. As of writing, the terminal-first agent harnesses in common use include Claude Code, OpenCode, Codex, Gemini CLI, Copilot CLI and Aider; IDE-embedded ones include Cline, Continue and Roo Code. Domain-specific agents exist for the life sciences: Biomni (Huang et al. 2025), PantheonOS (Xu et al. 2026), OmicClaw (Zeng et al. 2026), and others appearing monthly.

Names will change. The design — model, loop, tools, permissions, files — has been stable since the pattern emerged, and is the thing worth learning. Pick one that runs in your terminal, works with the models you can afford, and shows you what it is about to do. That is most of the decision.

1.5 Where agents are going

Two directions are worth watching because they bear on research practice directly.

The first is autonomous research. Lu et al. (2026) report a system that navigates the research process end to end — idea generation, experiment, writing — and Feng et al. (2026) describe a mathematics research agent that has produced genuine results. Song et al. (2026) automate manuscript production from raw materials. Qu et al. (2026) push toward multi-agent evolution for open-ended discovery. Whether this scales to field ecology, where the bottleneck is fieldwork rather than analysis, is a separate question.

The second is the benchmark reality check. Guo et al. (2026) evaluated bioinformatics agents across 244 expert-curated tasks and found performance declining sharply with task difficulty. The honest summary is that agents are useful for scoped, well-specified analytical work and unreliable for anything requiring sustained judgement. That gap is where a domain expert remains necessary, and it is not obviously closing.

2. Why Now?

The mathematics of neural networks is old. Backpropagation dates to the 1970s and 1980s; the perceptron to 1958. The question is not why neural networks work, but why they started working *this well* in the last few years. Three curves crossed at roughly the same time, and no one of them would have been sufficient.

2.1 Compute

GPUs were built to shade triangles. The operation they do fastest — dense matrix multiplication over many parallel lanes — happens to be the operation neural network training consists of almost entirely. That coincidence is the hardware story in one sentence.

What changed recently is not the coincidence but the scale of investment behind it. The framing has moved from product development to national industrial strategy (OpenAI 2026a), and the numbers have followed. Training runs that would have been institutionally unthinkable in 2015 are now routine at a handful of companies, and the capital expenditure numbers have moved into the range of national infrastructure programmes: the major cloud providers each committed to full-year capital outlays in the hundreds of billions of dollars through the 2026 cycle.

The consequence for everyone else is a two-tier world. Training a frontier model is out of reach for any university or research institute. Running one, on hardware you can buy, is not — and that gap is what Chapter 10 is about.

2.1.1 The memory wall

When you run a model on your own machine, the limit is not processor speed. It is memory: how much you have, and how fast you can read it. Model weights must be streamed through the compute units for every token generated, so tokens per second is governed by memory bandwidth far more than by raw FLOPs.

The rough sizing rule, widely used and accurate enough for planning:

Table 2.1: Memory required to hold model weights, by precision.

Precision	**Bytes per parameter**	**A 7B model needs**
FP16 (full)	2	≈ 14 GB
INT8	1	≈ 7 GB
Q4 (4-bit)	0.5–0.7	≈ 3.5–5 GB

Add roughly 20–30% for the key-value cache and runtime overhead, more if you want a long context window. A 32 GB machine comfortably runs a quantised 12B model with a 32K context; a 70B model at usable quality wants 48 GB or more.

So when sizing a machine, count memory first and processor speed last — the reverse of the habit most people bring from cluster computing. Chapter 11 works through the numbers.

2.2 Architecture

The transformer, introduced in 2017, removed the sequential bottleneck of recurrent networks. Attention lets every position see every other position in one parallel operation, which means the whole sequence trains at once rather than one step at a time. This made the compute curve usable: you could actually spend the GPUs you had.

Refinement continues. Kimi Team (2026) observe that standard residual connections accumulate every layer's output with fixed unit weights, which dilutes each layer's contribution as depth grows, and replace the fixed accumulation with learned attention over preceding layers. The pattern is typical: the architecture is not finished, but the changes are now incremental rather than paradigmatic.

2.3 Data, and who paid for it

The third curve is text. Pretraining consumed the accessible written output of the internet, and once that was exhausted the search moved to less accessible corpora — which is where the story becomes uncomfortable for people who care about libraries.

Several AI companies have acquired physical books in bulk, cut the spines off, scanned the pages destructively, and discarded the remains (James 2026). The economics are straightforward: destructive scanning is faster and cheaper than non-destructive scanning, and buying a book outright avoids a licensing negotiation. Volunteers have begun racing to scan rare material non-destructively before it enters that pipeline (Anna's Archive 2026).

For biodiversity this is not abstract. The Biodiversity Heritage Library holds digitised taxonomic literature going back to the eighteenth century — original species descriptions, type designations, expedition reports. Much of it exists in one or a handful of physical copies. The material is simultaneously the foundation of nomenclatural practice and an attractive training corpus, and the institutions holding it are not funded to defend it.

Key point. *Ecology's data commons was built by people, deliberately, over decades, to be shared. "AI-ready" describes what that work made possible. It does not describe who did it, or on what terms it is now being consumed.*

2.4 Convergence, and what it means for a field like ours

None of the three curves alone explains the moment. Cheap compute without the transformer gave you slow recurrent networks. The transformer without compute gave you a promising paper. Both without the corpus gave you a well-engineered model of nothing.

For ecology the relevant question is which of the three we actually have. Compute we can rent or buy. Architecture we get for free from the open-weight ecosystem. Data — domain data, curated, standardised, with provenance — is the one we own, and the one that determines whether any of this produces science rather than plausible text. That is the subject of Part II.

2.5 Is this a bubble?

It is a fair question, and the honest answer is that two things are being asked at once: whether the models work, and whether the money spent on them comes back. The capability is real and measurable: the AI Index (Stanford Institute for Human-Centered Artificial Intelligence 2026a) tracks steady benchmark improvement across domains. Whether the capital deployed returns to the firms deploying it is a question about business models, not about whether the models work.

Two knock-on effects reach research budgets either way. The first is hardware pricing: sustained demand for high-bandwidth memory has raised RAM and storage costs across the board, which affects every laboratory that buys a workstation. The second is dependency: infrastructure procured on the assumption of cheap API access becomes fragile if pricing changes, and the switching cost is paid by whoever built on top of it.

Keeping some capability on your own hardware is partly insurance against both.

3. Powers and Limitations

This chapter is the one to read if you read only one. The failure modes below are not bugs awaiting a patch. Several of them follow from how these systems are trained, which means they will be with us for a while.

3.1 What language models are genuinely good at

Stated plainly, so that the rest of the chapter is not mistaken for dismissal:

- **Transforming text whose content you supply.** Summarising, reformatting, translating between representations, converting prose to structured records. The input constrains the output, which is exactly the condition under which these systems are reliable.
- **Extraction from unstructured sources.** Castro et al. (2024) demonstrate this directly in ecology: natural history observations, invasion records and species interactions buried in free text can be extracted at scale and at accuracy that was not previously achievable.
- **Code.** Boilerplate, format conversion, plotting, regular expressions, reading unfamiliar codebases. Code is verifiable — it either runs and produces the right answer or it does not — which is why it is the strongest application area.
- **Getting started.** A bad first draft you can fix beats a blank page. This is a real effect and worth naming.
- **Explaining things you half-know.** Statistical methods, an unfamiliar file format, a compiler error. Verification is cheap here because you can usually tell when the explanation clicks.

The common thread: these are tasks where either you supply the ground truth or you can check the answer cheaply.

3.2 Hallucination

A language model asked a question it cannot answer will usually produce a fluent, plausible, specific and wrong answer. The technical term is *hallucination*; *confabulation* would be more accurate.

The important finding is that this follows from how the models are trained. Kalai et al. (2025) show that hallucinations originate as ordinary errors in binary classification — if incorrect statements cannot be distinguished from facts at training time, statistical pressure alone produces them — and, more consequentially, that they *persist* because of how evaluation works. Benchmarks score accuracy. A model that guesses when uncertain scores better than one that abstains. We have been optimising these systems to be good test-takers, and a good test-taker never leaves a blank.

Key point. *Hallucination is not a defect in an otherwise truthful system. It is what you get when you train for plausibility and grade on accuracy without rewarding abstention.*

3.2.1 The citation problem

The failure mode that matters most for research is fabricated references. Zhao et al. (2026) audited 111 million references across 2.5 million papers on arXiv, bioRxiv, SSRN and PubMed Central, and found a sharp rise in non-existent references coinciding with widespread LLM adoption — a conservative estimate of 146,932 hallucinated citations in 2025 alone.

These are not obvious. A hallucinated citation typically has real authors who work in the field, a real journal, a plausible year, and a title describing a paper that ought to exist. It passes a skim. It fails a lookup.

Retrieval-augmented generation (RAG) is the standard mitigation and it is only a partial one. Magesh et al. (2025) evaluated commercial legal research tools marketed as hallucination-free and found they hallucinate at substantial rates despite retrieval grounding. Kurniawan et al. (2024) catalogue seven distinct failure points in RAG pipelines — missing content, retrieval ranking failures, consolidation errors, extraction errors, wrong format, wrong specificity, incompleteness — any of which produces a confident wrong answer with a citation attached.

Key point. *Check every reference against a resolver — DOI, PubMed, OpenAlex — before it enters a manuscript. Not a sample. Every one. This is a five-minute scripted task and there is no good reason to skip it.*

3.3 Sycophancy

Models agree with you. Asked to evaluate your hypothesis, they find merit in it. Told they are wrong, they apologise and adopt your position, including when your position is the incorrect one.

Cheng et al. (2026) measured this across leading systems and found sycophancy widespread, with effects beyond the conversation: exposure to sycophantic AI *decreased participants' prosocial intentions* and increased dependence on the system. Chandra et al. (2026) model the mechanism and show that even an ideally rational Bayesian agent, updating correctly on evidence from a systematically validating source, will converge on increasingly confident false beliefs. The user need not be gullible. The input is biased, so the posterior is wrong.

This does most damage where you are least protected: late in an analysis, alone, with a result you want to be true. The system will help you want it more.

Practical countermeasures, in decreasing order of effectiveness:

- Ask for the argument against, explicitly and separately.
- Present your hypothesis as someone else's when asking for critique.
- Ask the same question in a fresh session without the conversational history that has been accumulating agreement.

- Do not use a chatbot as your only reader.

3.4 Dependence and cognitive offloading

Solé et al. (2026) model LLM adoption using a viral analogy: users transition among uncoupled, coupled and persistently dependent states, and the interplay between social transmission and collective reinforcement can produce tipping points and technological lock-in. Their result is a runaway dynamic — past a critical adoption threshold, small increases in use trigger rapid population-level shifts toward persistent dependence, with abrupt losses in the underlying competence.

The governance question this raises is genuinely hard, and the mental health case is where it has been examined most closely (Stanford Institute for Human-Centered Artificial Intelligence 2026d): the same properties that make a system engaging are the ones that make dependence likely, and the regulatory instruments available were not designed for either.

The framework also identifies the counter-condition, which they call *cognitive immunization*: reduce transmission, preserve reversibility. In practice that means keeping the skill you are offloading. If you no longer write a regression by hand, you are no longer positioned to notice when the generated one is wrong.

3.5 Provenance

A model output has no provenance. You cannot ask which training documents produced a claim, you cannot audit the derivation, and you cannot reproduce it exactly — sampling is stochastic, and the model behind an API endpoint changes without notice.

This is a direct conflict with how science establishes credibility. The chain from observation to claim is the thing that makes a result checkable. A generative step in that chain is an unlogged gap.

The mitigation is engineering, not prompting: log the prompt, the model identifier and version, the parameters, the retrieved context and the raw output, as plain text, in version control, alongside the analysis. Treat it as you would treat instrument settings. Chapter 9 develops this.

3.6 Jaggedness and the benchmark trap

Mitchell (2026) describes the pattern: capability is not a smoothly rising curve but a jagged frontier. Systems that reason through graduate-level problems fail at tasks a child handles, and — worse for planning purposes — the failures are not predictable from the successes.

The consequence is that benchmark scores do not transfer. Guo et al. (2026) evaluated bioinformatics agents on 244 expert-curated tasks and observed sharp performance decline with difficulty, with state-of-the-art agents clustering well below what headline numbers suggest. Turner (2026) makes the related observation for biosecurity evaluations: scores are highly sensitive to question format, and benchmark saturation has not been calibrated against real-world task performance.

A benchmark result tells you how a model performs on that benchmark. Test it on your own data, with your own task, or you do not know how it will do.

3.7 Context windows and what falls out of them

The context window is the maximum number of tokens a model can attend to at once. Everything the system knows in a given call is in that window: your question, the conversation so far, retrieved documents, file contents, tool results.

Tokens are subword chunks. Useful approximations for English:

- 1 token $\approx$ 4 characters
- 1 token $\approx$ 0.75 words
- 100 tokens $\approx$ 75 words

Code is more token-efficient than prose because it is repetitive. A 200-line R script runs roughly 1,000–2,000 tokens.

Two practical consequences. First, cost and latency scale with the window, so a long conversation is being re-sent in full on every turn. Second, and less obvious: attention degrades within the window. Material in the middle of a long context is attended to less reliably than material at either end. Filling a large window is not the same as using it.

So start a fresh session when the task changes. A short, relevant context beats a long one with the right answer buried in the middle of it.

3.8 A note on AI-generated text detection

Institutions increasingly want to detect AI-written text. The tools do not work reliably. Pudasaini et al. (2026) applied explainable AI methods to detectors reporting high benchmark accuracy and found they largely exploit dataset-specific artefacts rather than identifying machine authorship. Accuracy collapses out of distribution — and non-native English writing is systematically misclassified as machine-generated.

Do not use these tools to make decisions about people. The policy question (Chapter 13) is about disclosure, not detection.

4. Applications in Biology and Ecology

A survey of where these methods have actually produced results, and where the claims outrun the evidence. The distinction matters: the successes and the overreaches have a recognisable shape, and learning to tell them apart is worth more than memorising either list.

4.1 The pattern behind the successes

Structure prediction is the canonical case. AlphaFold worked because protein folding had the properties a deep learning problem needs: a large corpus of solved structures as ground truth, a well-defined input–output mapping, a physically constrained output space, and an unambiguous evaluation metric. The model learned a real regularity because a real regularity was there and was densely sampled.

Brixi et al. (2026) extend the same logic to genomes — modelling and design across all domains of life, at scale, from DNA sequence. Again: enormous training corpus, discrete well-defined alphabet, evaluable output.

Jiang et al. (2025) survey the broader field and report the landmark numbers: near-atomic protein structure prediction at median 0.96 Å on CASP14, protein design success rates up to 92%, sensitive cancer detection at AUC ≈ 0.93.

Key point. *Deep learning succeeds where there is a large, densely-sampled, well-labelled corpus and an unambiguous target. Ecological field data rarely has any of the three.*

4.2 Where ecology has genuine wins

4.2.1 Image classification

Camera traps, acoustic monitoring and remote sensing are the strongest cases, and for the same reason as structure prediction: large labelled corpora and an unambiguous target. Species identification from camera trap imagery routinely exceeds 90% accuracy for well-represented taxa while collapsing manual annotation time (Mishra 2026). Satellite imagery classification for forest cover change and vegetation type operates at spatial and temporal resolutions that were previously unattainable.

The caveats are the usual ones and they are not minor. Accuracy is reported on common species; rare species — the ones conservation actually cares about — are underrepresented in training data and classified worst. Models trained in one biogeographic region transfer poorly to another. And a classifier confident about a species outside its range is a data quality problem that propagates silently into downstream analysis.

4.2.2 Natural history collections

Johnson et al. (2026) report robotic imaging combined with AI-driven transcription of specimen labels at scale. This is a strong application because the bottleneck was always transcription throughput, not scientific judgement, and because the output is verifiable against the physical specimen. Roughly a billion specimens sit in the world's collections with label data locked in handwriting and typescript. Mobilising that is a mass-transcription problem, and mass transcription is what these systems do.

4.2.3 Text mining the literature

Biodiversity knowledge is disproportionately locked in prose — in taxonomic descriptions, expedition reports, faunal lists, and the grey literature. Castro et al. (2024) demonstrate LLM extraction from exactly this material.

Takola (2026) traces the methodological arc for conservation specifically: Latent Semantic Analysis to Latent Dirichlet Allocation to embedding-based models like BERTopic and Top2Vec to LLMs. Her conclusion deserves attention because it runs against the upgrade narrative: traditional probabilistic models remain valuable. They are interpretable, cheap, reproducible and auditable. An LLM topic assignment is none of those things. Newer is not automatically better when what you need is a method section you can defend.

Bibliometric synthesis is a related and undersold application. Ioannou et al. (2025) map One Health research across environmental, animal and human health dimensions in coastal and marine contexts — the kind of cross-domain landscape analysis that is tedious manually and tractable computationally.

Infrastructure is appearing for the retrieval side. Biodiversity PMC (Pasche et al. 2026) indexes biodiversity literature specifically. Krug and Ruch (2026) examine the role of AI in IPBES assessments and pose the question in their title — hype or salvation — without resolving it, which is the right answer for assessment work, where every statement has to be traceable to a source.

4.2.4 Microbiome and multi-omics

Yu et al. (2026) review the move from descriptive microbiome analysis toward predictive modelling and hypothesis generation, covering integrative analysis of microbial genomes, community structure and functional potential. They are explicit about what blocks the field: model interpretability, generalisability across cohorts, and causal inference. The first two are engineering problems. The third is not.

4.2.5 Curation support

PomBase and ELIXIR Curation Consortium (2026) describe the application closest to ELIXIR's own operations: AI-assisted literature curation for biological knowledgebases. Their framing is the correct one — *human-in-the-loop*, with generative methods accelerating routine tasks (recommendation, entity recognition, extraction, summarisation, ontology development, quality control) while curators retain critical evaluation. They identify the

risks precisely: scarce training data, difficulty extracting complex relationships, and error propagation into resources that other work depends on.

The last one is the reason this must stay human-supervised. A knowledgebase error does not stay local; it propagates into every analysis downstream, and attribution back to its source becomes impossible once it has been cited.

4.2.6 Query generation

Smeros et al. (2025) address a real friction point: writing SPARQL against federated life science knowledge graphs is a specialist skill that excludes most domain scientists. Their system generates queries from natural language questions and — unusually, and to their credit — evaluates on federated query capability, runtime and cost rather than accuracy alone.

This pattern generalises well. Generating a query is safe in a way that generating an answer is not: the query is inspectable before execution, the database provides ground truth, and a wrong query usually fails loudly. Prefer query generation over answer generation wherever the data is in a queryable store.

4.3 Where the claims outrun the evidence

Mishra (2026) makes the case directly, and the argument is worth stating in full because it is the most important dissent in this literature. Ecological data is heterogeneous, spatially and temporally biased, sparse for the taxa that matter most, and generated by processes that are causal rather than merely correlational. A model that predicts species occurrence from environmental covariates learns the sampling effort as readily as the ecology. Deploying such models for conservation decisions transfers the bias into policy while adding a veneer of quantitative authority.

The specific hazards:

- **Sampling bias passed off as signal.** Occurrence data records where people looked. A model trained on it predicts where people looked.
- **Absent causal structure.** Correlation between a covariate and a distribution is not a mechanism, and extrapolating to novel climate conditions requires the mechanism.
- **Uncertainty that does not propagate.** Deep learning outputs are typically point predictions. Ecological inference needs intervals, and downstream decisions need them honestly propagated.
- **Unfalsifiable models.** An uninterpretable model that fits well cannot be distinguished from one that fits well for the wrong reason.

Key point. *Before applying a model to an ecological question, ask what would have to be true of the data for the answer to be valid — and whether it is. The question is not whether the method is powerful. It is whether the data can support the inference.*

4.4 Agents in the life sciences

A distinct category: not models that classify, but agents that run analyses. Biomni (Huang et al. 2025) is a general-purpose biomedical agent; PantheonOS (Xu et al. 2026) a multi-agent framework for genomics discovery; OmicClaw (Zeng et al. 2026) targets executable and reproducible natural-language multi-omics analysis. Domain-specific skill libraries are emerging alongside them.

The evaluation literature is the useful part. Guo et al. (2026) benchmarked these systems on 244 expert-curated bioinformatics tasks and found comparable performance between leading agents and sharp decline with difficulty. The honest reading: useful for scoped, well-specified analytical tasks; unreliable for anything requiring sustained judgement about whether the analysis is the right one.

Nor is the randomness solved. Two runs of the same agent on the same input can give different answers. If the workflow has to be reproducible, that is a serious problem rather than a rough edge — see Chapter 9.

4.5 Domain assistants

UNEP's EnvironmentGPT (United Nations Environment Programme 2026) is instructive as a design pattern: a RAG system restricted to a curated library of UNEP and partner publications, rather than a general model over uncontrolled web text. The tradeoff is explicit — narrower coverage in exchange for citable answers and reduced hallucination.

Sigillo et al. (2026) generalise this into infrastructure: a life-sciences knowledge layer that AI agents query, rather than each agent scraping and indexing independently. This is the right architecture. The alternative, in which every group builds its own RAG pipeline over the same literature, wastes effort and produces inconsistent answers to identical questions.

For an assistant of this kind, a curated set of documents with citations beats a larger model without them. The value is in the documents and in being able to check where an answer came from, not in the model.

Part II

Data

5. AI-Ready Data

For several decades, biodiversity data has been published openly. Occurrence records, sequences, taxonomies, trait databases, ecological time series — curated, standard-compliant, documented, and given away. This was the product of sustained, unglamorous, badly-funded work by people who believed that shared data compounds.

AI-ready arrived as a description of what that work made possible. The term is worth examining, because it is doing something other than describing a technical property.

5.1 What the term conceals

Data is called AI-ready when it is machine-parseable, uses controlled vocabularies, carries structured metadata, is available in bulk without per-record negotiation, and is licensed permissively. Every one of those properties is a description of good data management, which is what the biodiversity informatics community has been doing since well before the term existed.

What the term adds is a consumer. And the consumer is frequently a commercial entity that will ingest the corpus, produce a model, and release neither the model weights nor any attribution to the curators. Decades of public and volunteer labour flow one way; a product under restrictive terms flows back, if anything does.

This is not an argument against openness. Open data has been enormously productive for ecology and the alternative is worse. It is an argument for paying attention to terms.

Key point. *When a funder or partner asks you to make data "AI-ready", the useful follow-up questions are: ready for whom, under what licence, with what attribution, and does anything come back to the people maintaining it?*

5.2 Standards worth knowing

Standards are what make data useful to anything other than the person who produced it. The ones that matter here fall into layers.

5.2.1 Data description

Darwin Core remains the backbone of biodiversity occurrence exchange, with Darwin Core Archive as the packaging format. ABCD covers collection and observation data. MIxS specifies minimum information for sequence data — critical for environmental omics, and the reason a metagenome from a Greek coastal site can be compared against one from the North Sea.

Ontologies do the semantic work: ENVO for environments, EnvThes and EUNIS for habitat classification, the Plant Ontology, the Gene Ontology and its relatives. Choosing between

overlapping vocabularies — ENVO versus EUNIS versus LUCAS for a habitat field — is a real design decision with downstream consequences, not a formality.

Magagna et al. (2021) address the layer above: I-ADOPT provides an interoperability framework for variable descriptions, decomposing a measured variable into its components so that "sea surface temperature" from two networks can be recognised as the same thing. This is the sort of infrastructure that pays off invisibly and is invisible when it is missing.

5.2.2 Machine interfaces

- **POSIX** — the file and process abstractions everything else assumes. Unglamorous, and everything rests on it (Ritchie and Thompson 1974).
- **OpenAPI** — machine-readable HTTP API description. If your service has an OpenAPI specification, an agent can discover and call it without custom integration.
- **MCP (Model Context Protocol)** — a standard for exposing tools and data sources to language models. The practical significance is that a resource needs to be wrapped once rather than once per client. Treat it as infrastructure, and read Chapter 12 before exposing anything sensitive through it.
- **SPARQL and RDF** — for the knowledge graph layer. Smeros et al. (2025) show that LLM query generation over these is now practical, which changes the accessibility calculus for federated resources.

5.2.3 Data harmonisation in practice

Gill et al. (2023) describe a tool for exactly the problem that consumes most of the time in any integration project: taking contextual metadata in whatever shape it arrived and validating it against a specification. This work is not interesting and it is the difference between a dataset that can be combined with others and one that cannot.

5.3 Infrastructure

5.3.1 Biodiversity platforms

GBIF aggregates occurrence records globally. OBIS does the marine equivalent. PlutoF (Kõljalg et al. 2019) manages the complete data lifecycle from collection through publication, with UNITE for fungal sequence identification built on the same foundations. BOLD holds barcode sequences; ENA, GenBank and SRA hold the rest.

The pattern to notice: these are institutional, long-lived, and funded through research infrastructure programmes rather than commercially. Their persistence is a policy achievement, not a market outcome, and it is not guaranteed.

5.3.2 Model and dataset platforms

Hugging Face has become the default distribution point for open-weight models and, increasingly, datasets. Convenient, and worth understanding as a dependency — it is a single commercial point of failure for a large part of the open model ecosystem.

It is also an attack surface. OpenAI (2026b) describe a security incident arising during model evaluation, and the general problem is structural: downloading and executing a model artefact from a public repository is code execution from an untrusted source. Pickle-format weights can carry arbitrary payloads. Prefer safetensors; verify checksums; treat a model download with the same suspicion you would give a random binary.

5.3.3 Standards and governance bodies

The argument for open standards in scientific publishing predates this cycle and applies to it unchanged (Wilson and Tchantchaleishvili 2013): formats and tools that anyone can implement are the ones that outlive the organisations that introduced them.

The LF AI & Data Foundation hosts a number of the projects underpinning open model tooling. TDWG (Biodiversity Information Standards) governs Darwin Core and its relatives. ELIXIR coordinates life science data infrastructure across Europe and maintains the Core Data Resources designation that marks which databases the community has committed to sustaining.

5.4 Open weights, open source, and the difference

The distinction is frequently blurred and it matters.

Open weight means the model parameters are downloadable. You can run it, fine-tune it, and inspect its outputs. You cannot reproduce it, audit what it was trained on, or verify claims about its data.

Open source, applied properly, would require the training data, the training code, the data processing pipeline, and the weights — everything needed to reproduce the artefact.

Almost every model marketed as open is open-weight. Stanford Institute for Human-Centered Artificial Intelligence (2026c) argue this is insufficient for science and society: without training data disclosure you cannot assess contamination, audit for bias, verify that your evaluation set was not in the training corpus, or reproduce the result. For a scientific instrument, that is a serious gap.

Licences vary more than the labels suggest. Apache-2.0 and MIT are genuine open source licences. Several prominent "open" model licences impose use restrictions or downstream conditions that do not meet the Open Source Definition. Read the licence before building anything you intend to maintain.

Table 5.1: Degrees of openness. Only the last row supports reproduction.

Level	**You get**	**You cannot**
API only	An endpoint	Run offline; pin a version; audit anything
Open weight, restricted licence	Weights, conditional use	Use freely commercially; audit training data
Open weight, Apache/MIT	Weights, unrestricted use	Audit training data; reproduce
Fully open source	Weights, data, code, pipeline	—

5.5 Retrieval-augmented generation

RAG is the main technique for giving a model access to knowledge it was not trained on, without retraining. The mechanism is simple enough to state in a paragraph.

Ask a model "what colour is my ball?" and it will invent an answer. Say "I have a green box with a red ball in it — what colour is my ball?" and it will answer correctly. RAG automates the insertion of that second clause: a retrieval system searches a document store for material relevant to the question and prepends it to the prompt.

The pipeline:

1. **Ingest** — convert source documents to text. For PDFs this is the hardest step and the quality ceiling for everything downstream. Auer et al. (2024) describe Docling, an MIT-licensed converter using layout analysis and table structure recognition that runs on commodity hardware. Worth using over naive extraction.
2. **Chunk** — split into passages. Too small loses context; too large dilutes the embedding. Respect document structure where you can.
3. **Embed** — map each chunk to a vector. `bge-m3` (MIT) and `nomic-embed-text` (Apache-2.0) are solid open choices.
4. **Index** — store vectors with metadata. PostgreSQL with `pgvector` keeps text, embeddings and metadata in one ACID database and scales to around a million documents comfortably. Qdrant is the dedicated alternative when throughput demands it.
5. **Retrieve** — hybrid search (full-text plus vector) outperforms either alone. Rerank with a cross-encoder such as `bge-reranker`.
6. **Generate** — pass retrieved context plus question to the model, requiring citation of the retrieved passages.

5.5.1 Where it breaks

Kurniawan et al. (2024) identify seven failure points, each of which produces a confident wrong answer:

1. Missing content — the answer is not in the corpus.
2. Missed top-ranked documents — it is there, retrieval ranked it low.
3. Not in context — retrieved, but truncated out of the window.
4. Not extracted — present in context, the model missed it.
5. Wrong format — the output does not match the requested structure.
6. Incorrect specificity — too general or too narrow to be useful.
7. Incomplete — partially correct, missing material qualifications.

Note that failures 1–3 are retrieval problems and 4–7 are generation problems. They need different instrumentation. Log what was retrieved, separately from what was generated, or you cannot tell which half is broken.

5.5.2 An alternative worth considering

RAG is not the only option. For a bounded corpus, compiling the knowledge into a structured document — a wiki, a set of curated notes — and placing it in context directly can outperform vector retrieval, because it removes the retrieval failure modes entirely. This works when the corpus is small enough to fit and stable enough to be worth compiling. It is worth benchmarking against a RAG pipeline before assuming retrieval is required.

5.6 Provenance as a first-class requirement

Everything in this chapter converges on one point. If a model-assisted result is going to stand up to scrutiny, the path from source to claim has to be written down.

- Retrieved passages, with document identifiers and versions.
- Model identifier and version string.
- Generation parameters, including temperature and seed where available.
- The raw output, before editing.
- The date. Hosted models change without announcement.

Plain text, in the repository, next to the analysis. This is the same discipline as recording instrument calibration, and it is not optional if the result is going into a paper.

6. Organising and Protecting Your Data

This chapter is about filing and backups. It is in a book about AI because agentic workflows amplify both the value of good organisation and the cost of bad organisation, and because an agent with write access to a badly-backed-up directory is a new category of risk.

6.1 Why this matters more now

An agent navigates your project by reading it. A directory tree with meaningful names, a `README.md` that explains the structure, and consistent file naming lets an agent locate the right file in one or two operations. A tree named `final/`, `final2/`, `new_final/`, `Untitled1.csv` forces it to read everything, burn context on irrelevant material, and guess.

The second reason is blunter. Agents delete files. Sometimes because you asked them to, sometimes through a misread instruction, occasionally through a bug. Shapira et al. (2026) document execution of destructive system-level actions among the failure modes observed when autonomous agents were given real file system and shell access in a live environment. Your backup strategy is the thing standing between that and a lost field season.

6.2 PARA

PARA is a general-purpose organisation method that adapts well to research. Four top-level categories, by *actionability* rather than topic:

Projects
Active work with a defined outcome and an end date. A manuscript in preparation, a specific analysis, a deliverable.

Areas
Ongoing responsibilities with no end date. A long-term monitoring programme, a database you maintain, teaching.

Resources
Reference material of future interest. Literature, protocols, reusable code, notes.

Archive
Anything from the first three that is finished or dormant.

The mechanism that makes it work is the archive. When a project closes it moves to `Archive/` intact. The active tree stays small, which keeps it navigable — for you and for an agent.

Listing 6.1: A research tree, PARA-shaped

```
~/work/
  10_projects/
    2026_posidonia_edna/
      README.md          # what, why, status, how to run
      AGENTS.md          # conventions for agents working here
```

```
      data/
        raw/              # never edited, read-only, checksummed
        processed/        # derived; regenerable from raw + scripts
      scripts/
      results/
      manuscript/
  20_areas/
    monitoring_network/
    teaching/
  30_resources/
    literature/
    protocols/
  40_archive/
    2024_thermal_tolerance/
```

The numeric prefixes keep sort order stable and make paths easy to type and to reference in conversation.

6.3 Naming

Three rules cover most of it.

Sortable dates. `YYYY-MM-DD` or `YYYYMMDD`. These sort lexicographically into chronological order, which no other format does.

No spaces or special characters. Underscores between fields, hyphens within them. Spaces break shell pipelines, require quoting everywhere, and produce a steady trickle of bugs in scripts written by humans and agents alike.

Machine-readable structure. Encode the metadata in the name, in consistent field order:

```
20260415_crete-north_station03_16S_rep1.fastq.gz
20260415_crete-north_station03_16S_rep2.fastq.gz
^date     ^site        ^station  ^marker ^replicate
```

This is greppable, globbable, and parseable into a data frame with one split.

A name a script can select and split is worth more than a descriptive sentence. Put the description in the README.

6.4 Raw data is read-only

The single most valuable convention in this chapter: raw data is never modified. Set it read-only, checksum it, and derive everything else.

```
chmod -R a-w data/raw/
find data/raw -type f -exec sha256sum {} + > data/raw/CHECKSUMS.sha256
sha256sum -c data/raw/CHECKSUMS.sha256      # verify later
```

Everything in `processed/` should be reconstructible by running the scripts against `raw/`. If it is not, that is a gap in the pipeline, and it is better to find out now than during review.

This convention also protects you from agents. An agent instructed to clean a dataset will happily clean it in place. Read-only permissions turn that into a visible error rather than a silent loss.

6.5 Backup: the 3-2-1 rule

Three copies of the data, on two different media, with one off-site.

- **Copy 1** — the working copy on your machine.
- **Copy 2** — a local backup on different physical media. An external drive, a NAS. Different media matters: a second partition on the same disk dies with the disk.
- **Copy 3** — off-site. Institutional storage, cloud sync, a drive at another building. This is the copy that survives fire, theft and flood.

6.5.1 RAID is not a backup

RAID protects against *drive failure*. It does not protect against accidental deletion, ransomware, filesystem corruption, controller failure, a power event, theft, or an agent running `rm -rf` against the wrong path. Every one of those propagates instantly to every disk in the array.

A NAS with RAID is a good copy 2. It is not copies 2 and 3, and treating it as both is the most common backup mistake in research computing.

6.5.2 Verify restores

An unverified backup is a hypothesis. Schedule a restore test — pick a random file, restore it from each copy, diff it against the original. Quarterly is enough. The failure you are testing for is the one where backups have been silently failing for eight months.

6.6 Version control

Git for code, always. For a project that involves agents, it stops being optional: `git diff` is how you review what an agent actually changed, `git checkout` is how you undo it, and commit granularity is how you bisect when something breaks three steps later.

Listing 6.2: Minimum viable discipline

```
git init
git add README.md AGENTS.md scripts/ .gitignore
git commit -m "Initial structure"

# commit before letting an agent loose
git commit -am "checkpoint before refactor"
# ... agent works ...
git diff                    # review every line
git checkout -- .           # or undo the lot
```

Large data files do not belong in Git. Use Git LFS, DVC, or — most often the right answer — keep data out of the repository entirely and record its location and checksum in the README. Zenodo gives you a DOI and long-term storage for the archival copy.

6.7 Plain text lasts

A closing principle that runs through the rest of this book. The formats that survive are the simple ones: plain text, Markdown, CSV, TSV, and well-specified binary formats with open readers such as Parquet and NetCDF.

Plain text is greppable, diffable, version-controllable, readable by every tool, and legible in fifty years without a specific application. It is also the format agents work with best, which is not a coincidence — they were trained on it.

Proprietary formats, database-internal state and application-specific exports are where data goes to become inaccessible. When a system offers you a choice, take the text.

Part III

Working with Agents

7. The Agentic Coding Workflow

The change this chapter describes is not that code gets written faster. It is that the job changes. You stop being the person who writes the code and become the person who specifies it, constrains the environment it runs in, and verifies what came out. That is closer to system administration than to programming, and the skills that transfer are the operational ones: permissions, logging, reproducibility, knowing what you would do if it went wrong.

7.1 Terminology, briefly

Vibe coding was coined in early 2025 for accepting AI-generated code without close review — going with the flow. It has since been diluted into shorthand for any AI-assisted coding, which loses the distinction.

Chat-oriented programming (CHOP) is the more useful term: an iterative loop in which you describe intent, the system proposes code, you run it, and you refine. The distinguishing feature is that you read and verify at each step.

The difference is whether verification is in the loop. Vibe coding is fine for a throwaway plot. It is not fine for anything entering a manuscript, a database, or a pipeline someone else will run.

Key point. *The question is not whether AI wrote the code. It is whether anyone checked it. You are accountable for what you publish regardless of what produced it.*

7.2 Architecture: chatbot versus harness

Table 7.1: Two interaction models, two risk profiles.

	Chatbot	**Harness**
Interface	Web or app	Terminal, in your project
Sees	What you paste	Your files, directly
Acts	No	Runs commands, edits files
Verification	You copy, paste, run	It runs, reads the error, retries
Failure cost	A wrong snippet	Modified or deleted files
Good for	Explaining, drafting	Multi-file work, refactors, pipelines

The capability difference comes from the feedback loop. A harness sees the traceback. A chatbot sees what you chose to paste, which is usually the last five lines, which is usually not where the problem is.

7.3 The while loop

Liu et al. (2026) analysed a production coding agent's source and found the core to be exactly the loop from Chapter 1: call the model, execute requested tools, feed results back, repeat until done. Their analysis traces five motivating values — human decision authority, safety and security, reliable execution, capability amplification, contextual adaptability — through thirteen design principles to implementation choices.

The distribution of code is the finding worth internalising. Almost none of it is the loop. The bulk is the permission system, context management, error recovery and memory. When you evaluate a harness, evaluate those, because that is what differs.

7.4 A file is all you need

Piskala (2026) makes the Unix argument for agent design: collapse heterogeneous resources into uniform file-like interfaces. In practice this means your project communicates with an agent through files, and those files are readable by you.

7.4.1 README.md

Written for a human, read by both. It should state what the project does, how to run it, what the data is and where it lives, what the dependencies are, and what the current state is. If a new postdoc could not start from it, an agent cannot either.

7.4.2 AGENTS.md

This is the behavioural contract. Not aspirations — constraints, written in response to failures you have observed.

Listing 7.1: An AGENTS.md that earns its place

```
# Conventions

## Environment
- R 4.4 via renv. Activate with `renv::restore()`.
- Never install packages globally. Add to renv.lock.

## Data
- data/raw/ is READ-ONLY. Never write there, never modify.
- All derived data goes to data/processed/ and must be
  regenerable by running scripts/ in numeric order.
- Coordinates are WGS84 decimal degrees. Do not reproject
  silently; if a reprojection is needed, say so first.

## Code
- Tidyverse style. snake_case. No magic numbers.
- Every script starts by reading from data/ and ends by
  writing to results/. No side effects elsewhere.
- Taxon names go through taxonomic backbone matching before
  any join. Never join on raw name strings.
```

```
## Don't
- Don't commit. I commit.
- Don't touch manuscript/ unless explicitly asked.
- Don't add dependencies without asking.
```

Each line there closes a specific failure. The taxon name rule exists because joining on unmatched names silently drops records. The reprojection rule exists because a silent CRS change produces a map that looks fine and is wrong.

7.4.3 Skills and packaged instructions

Several harnesses support bundling instructions, scripts and reference material into reusable units — variously called skills, plugins or commands (Anthropic 2026). The useful ones are, again, just directories of files: a description of when to use them, the procedure, and any scripts needed. For a laboratory with a standard analysis protocol, packaging it this way is a reasonable way to make the protocol executable rather than merely documented.

7.4.4 Memory

Some harnesses maintain persistent memory across sessions. When they do, prefer the implementations that store it as files you can read and edit. Opaque memory is a reproducibility problem: if the agent behaves differently today and you cannot see why, you have lost control of your own environment.

7.5 Tokens, context and cost

Tokens are the unit of both billing and context. The approximations from Chapter 3: one token is about four characters or three-quarters of a word; a 200-line R script is roughly 1,000–2,000 tokens.

Everything in the context window is re-sent on every turn. A long session with a large file loaded is paying for that file repeatedly. Subscription plans meter this: a typical consumer tier allocates on the order of tens of thousands of tokens per rolling five-hour window, which a single large refactor can consume.

Practical economy:

- Start a new session per task. Do not carry an hour of unrelated history.
- Point at files rather than pasting them; let the agent read what it needs.
- Ask for a diff, not a rewritten file.
- Use a smaller model for mechanical work. Not everything needs the frontier model.
- Keep a scratch file for intermediate results instead of keeping them in context.

7.6 Prompting, practically

Prompt engineering as a discipline is overstated for frontier models and genuinely important for smaller local ones. What consistently helps:

State the goal, the constraints, and the definition of done. "Fix the script" produces guesswork. "The script fails at line 40 with this traceback; it should output one row per station with mean abundance; do not change the input format" produces a fix.

Provide an example. One worked input–output pair beats three paragraphs of specification, particularly for data transformation.

Ask for a plan first on anything non-trivial. Reviewing a plan costs a minute. Reviewing a wrong 400-line implementation costs an hour.

Say what not to do. Negative constraints are effective and underused. "Do not modify data/raw. Do not add dependencies. Do not reformat unrelated code."

Give it a way to check itself. A test, an assertion, an expected row count. An agent that can verify will iterate to correctness; one that cannot will report success.

7.7 The review discipline

The failure mode that matters is not bad code. It is plausible code that runs, produces output of the right shape, and is subtly wrong.

1. **Commit before.** Always. A clean checkpoint makes everything reversible.
2. **Read the diff.** All of it. If it is too large to read, the task was too large — revert and split it.
3. **Check the boundaries.** Empty inputs, single rows, missing values, duplicate keys. These are where generated code fails.
4. **Verify the joins.** Row counts before and after. Silent row loss in a merge is the single most common serious error in ecological data processing, and agents produce it readily.
5. **Run it twice.** Same input, same output. If not, there is hidden state or unseeded randomness.
6. **Sanity-check the science.** Does the magnitude make sense? Are the units right? Would you have expected this? The agent has no domain intuition and will not flag a biologically impossible result.

Key point. *Agents fail loudly on syntax and quietly on semantics. Your review should spend its time on the second kind.*

7.8 Scope control

The most reliable determinant of success is task size. Agents are good at bounded, well-specified tasks and degrade sharply as scope grows — which is what Guo et al. (2026) measured across 244 bioinformatics tasks.

Good scope: “write a function that reads this CSV and returns a tidy data frame with these columns.” Bad scope: “build the analysis pipeline.” The second is a sequence of the first, and running it as a sequence gives you a review point between each step.

8. The Software Stack

A map of the layers you will actually touch, and the choices that are worth making deliberately.

8.1 Layers

Operating system
Linux is the path of least resistance: the tooling targets it first, containers are native, and the permission model is the one every guide assumes. macOS is a close second and has a specific advantage for local inference (Chapter 11). Windows works through WSL2, which is Linux with extra steps.

Language and libraries
R for ecological statistics and spatial work; Python for machine learning and general glue. Most projects end up using both, and that is fine.

Environment management
`renv` for R, `uv` or `conda` for Python. Essential if the work needs to run more than once.

Containers
Podman or Docker for the hard reproducibility cases and for sandboxing agents. Podman is rootless by default, which matters here.

Editor and harness
Positron, RStudio, VS Code, Neovim — plus whichever agent harness you settle on.

Version control
Git, plus a forge. GitHub, GitLab, or an institutional instance.

Workflow engine
`targets` for R, Snakemake or Nextflow for multi-tool pipelines. These give you dependency tracking and selective re-execution, which matters more as pipelines grow.

8.2 Interfaces

Four ways to talk to a system. A graphical interface is easy to explore and impossible to script. A terminal interface works over SSH and takes longer to learn. A command line composes, scripts and logs, if you know the commands. An API does anything, if you write the code.

For reproducibility the command line wins, and not narrowly. A command is a record of what you did; a sequence of clicks is not. This is why agent harnesses are terminal-first — the same property that makes a CLI reproducible makes it automatable.

8.3 Ten rules for code that survives

Collected from practice and from Nagraj et al. (2026), whose SNIFF framework — Source, Network, Interaction, Fit, Fragility — is aimed at the under-served question of how to evaluate software you are about to *depend on* rather than software you are writing.

1. **Minimise dependencies.** Every package is a maintenance liability and a supply chain risk. Prefer base and well-established libraries. Ask whether a dependency earns its place.
2. **Pin versions.** `renv.lock`, `uv.lock`, `requirements.txt` with exact versions. "It worked last month" is not a state you can return to otherwise.
3. **Version control everything textual.** Code, configuration, documentation, parameter files, notes.
4. **One script, one job.** Numbered, in order, each reading from defined inputs and writing to defined outputs. No hidden state between them.
5. **No absolute paths.** Relative to project root. `here::here()` in R, `pathlib` in Python.
6. **Set seeds.** Everywhere there is randomness. Record them.
7. **Assert your assumptions.** Row counts, value ranges, no unexpected `NA`s, join cardinality. Failing loudly beats propagating silently.
8. **Write the README as you go.** Written afterwards, it is written badly or not at all.
9. **Automate the whole run.** One command from raw data to figures. If you cannot, you do not know that it works.
10. **Evaluate before adopting.** Apply SNIFF: who maintains it and why, is it part of a wider ecosystem, are developers and users active, does it fit the use case and licence requirements, how fragile are its dependencies?

Working with agents raises the payoff on all ten. An agent in a well-organised project is fast and safe. The same agent in a disorganised one is fast and dangerous.

8.4 Local model tooling

8.4.1 Runtimes

Ollama is the standard on-ramp. It packages weights, configuration and parameters into a Modelfile, serves an OpenAI-compatible API on `localhost:11434`, and handles the chat templating. **llama.cpp** sits underneath much of this and is what you use when you need control. **vLLM** targets throughput for multi-user serving. **LM Studio** is the GUI option. On Linux, running the runtime in a rootless container is straightforward and worth doing — Monshed (2026) give a working Podman recipe on Fedora, which also solves the GPU passthrough question that otherwise consumes an afternoon.

8.4.2 Interfaces

Open WebUI gives you a familiar chat interface over one or more backends, local and remote, switchable per conversation. **text-generation-webui** is the more configurable alternative, with an extension system.

The extension mechanism is more interesting than it first appears. Extensions can rewrite what goes into the model and what comes out. Jensen's *Seshat*, for instance, enriches a prompt with material retrieved from PubMed before it reaches the model, and post-processes the output to strip inflated language. That is a domain-specific RAG layer implemented as a plugin, and the pattern generalises to any field with a good literature API.

8.4.3 A working configuration

Listing 8.1: Local coding setup with Ollama and OpenCode

```
# 1. Pull a coding model
ollama pull gemma4:12b

# 2. Bake in a usable context window.
#    The 4K default silently breaks agent tool loops.
ollama run gemma4:12b
  /set parameter num_ctx 32768     # 65536 if RAM allows
  /save gemma4:12b-32k
  /bye

# 3. Verify while the agent is running
ollama ps        # CONTEXT must read 32768, not 4096
```

Listing 8.2: opencode.json — the two things that must be right

```
{
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "options": { "baseURL": "http://localhost:11434/v1" },
      "models": {
        "gemma4:12b-32k": { "name": "Gemma 4 12B", "tools": true }
      }
    }
  }
}
```

Two failure modes account for most local-agent problems. Without `"tools": true` the model stalls on its first tool call. Without the enlarged `num_ctx` the agent loop truncates and behaves erratically in ways that look like model incompetence.

Set expectations accordingly: a 12B local model is genuinely useful for scoped edits, explanation and boilerplate, and well behind frontier models on multi-file reasoning. Drive it carefully and keep tasks small.

8.5 Speech-to-text

Worth a section because interview and field-recording transcription is a real need and the tooling is mature.

`whisper.cpp` runs Whisper locally with no API dependency. On Apple Silicon it can use the Neural Engine via Core ML for roughly a threefold speedup over CPU.

```
git clone https://github.com/ggml-org/whisper.cpp
cd whisper.cpp
sh ./models/download-ggml-model.sh large-v3    # best multilingual

cmake -B build                                  # -DWHISPER_COREML=1 on macOS
cmake --build build -j --config Release

# 16-bit WAV only; convert first
ffmpeg -i input.mp3 -ar 16000 -ac 1 -c:a pcm_s16le out.wav

./build/bin/whisper-cli -m models/ggml-large-v3.bin \
    -f out.wav -l el -otxt --print-colors --threads 7
```

8.5.1 A failure mode worth knowing

Whisper hallucinates on silence. On quiet passages, music, or background noise it emits plausible text that was never spoken — most notoriously the caption `AUTHORWAVE`, an artefact of subtitle data in the training corpus.

This is documented in the corpus-building literature. The Greek Podcast Corpus team, transcribing with WhisperX, removed segments containing the "Ypotitloi AUTHORWAVE" caption ("AUTHORWAVE subtitles") and, separately, segments with code-switched English–Greek phrases where the monolingual phoneme aligner produced bad timestamps. Between voice-activity filtering and post-processing they discarded roughly 20% of their data.

The lesson transfers: run voice activity detection first, filter known hallucination markers, and spot-check. Never treat an automatic transcript as ground truth without review — particularly for interview material, where an invented sentence attributed to a named person is a serious problem.

9. Reproducibility When the Tool Varies

Reproducibility has a precise vocabulary that is routinely used loosely. Table 9.1 sets it out. Then we deal with the problem this book introduces: a step in the pipeline that does not give the same answer twice.

9.1 The vocabulary

Table 9.1: Terms of art, and what each one actually assesses.

Term	Definition	Held constant	Varied	Assesses
Repeatability	The same team produces the same result under identical conditions	Team, data, code, environment	Nothing	Internal consistency
Reproducibility	An independent team produces the same result from the same data and code	Data, code	Team, environment	Computational integrity
Replicability	Independent teams reach comparable conclusions with new or independent data	Question, methodology	Team, infrastructure, data	Scientific validity
Reusability	Data or code can be repurposed for different ends	Digital assets	Purpose, context	Long-term usefulness
Provenance	Documented origin and processing history	Metadata, logs	Execution snapshot	Transparency, auditability

Most claims about reproducible research are really claims about repeatability. The gap between them is the computational environment, which is why containers and lockfiles matter.

9.2 Where a language model breaks this

Five distinct problems, with different solutions.

9.2.1 Sampling is stochastic

Generation samples from a distribution. Two calls with identical inputs produce different outputs unless temperature is zero and the seed is fixed — and even then, batching and hardware differences in floating-point reduction order can change results.

Mitigation: set temperature to 0 and fix a seed where the API exposes one. Accept that this gives you approximate determinism, not exact. Record the settings.

9.2.2 The model changes underneath you

A hosted endpoint named `model-4` is not a fixed artefact. Providers update weights, adjust system prompts and change safety filters without version bumps. Code written against it last year may behave differently today, with nothing in your repository to explain why.

Mitigation: this is the strongest single argument for open-weight local models in research. A local model file with a known checksum does not change. If you must use a hosted model, record the full version identifier and the date, and expect the result not to reproduce.

9.2.3 There is no provenance for a generated claim

You cannot trace an output back to its source. There is no derivation to audit.

Mitigation: keep the model out of the inferential path. Use it to generate code, which you then read and run — the code is the artefact, and it is deterministic and reviewable. Use it to generate a query, which you inspect before execution. Do not use it to produce a number that goes into a result.

Key point. *Generated code is reproducible. Generated answers are not. Push every model-dependent step as far upstream as it will go, and make the deterministic artefact the thing you keep.*

9.2.4 Agents take different paths

Given the same task, an agent may read different files, run different commands, and arrive at a different implementation. Shapira et al. (2026) observed something worse in a live deployment: agents reporting task completion while the underlying system state contradicted the report.

Mitigation: the artefact is the committed code, not the session. Review the diff, commit it, and treat the transcript as a log rather than as the deliverable. Never accept a completion report without checking the state yourself.

9.2.5 Context is invisible state

The same prompt at different points in a conversation produces different results, because the accumulated context differs. This is genuine hidden state and it is not recorded anywhere.

Mitigation: for anything that matters, use a fresh session with an explicit, self-contained prompt — and save the prompt.

9.3 A logging convention

Treat model calls as instrument readings.

Listing 9.1: logs/2026-04-15_taxon-matching.md

```
# Taxon name matching, first pass
Date:        2026-04-15
Model:       qwen3-coder:30b (local, ollama)
Checksum:    sha256:4f2a...c91
Temperature: 0
Seed:        42
Context:     fresh session, no prior turns
Harness:     opencode 0.4.2

## Prompt
Given data/raw/species_list.csv with column ‘verbatim_name‘,
write an R script that matches names against the GBIF backbone
using rgbif::name_backbone_checklist and writes
data/processed/matched_names.csv with columns:
verbatim_name, matched_name, rank, status, confidence.
Do not drop unmatched rows; mark them status="NO_MATCH".

## Output
-> scripts/03_match_taxa.R  (committed: a3f9c21)

## Review notes
- Checked: 1,847 in, 1,847 out. No silent loss.
- 23 NO_MATCH, all inspected manually; 19 were typos, corrected
  in a lookup table; 4 are genuine nomina nuda.
- Confidence threshold NOT applied by the script. Deliberate:
  filtering is a downstream decision.
```

Plain Markdown, in `logs/`, committed. It takes two minutes and it is the difference between a methods section you can defend and a shrug.

9.4 What to write in a methods section

Journals increasingly require disclosure, and the policies are converging (Nature Methods Editorial 2026). State:

- Which tool, which model, which version, and when.
- What it was used for — code generation, text extraction, language editing, literature search — with specificity.
- What verification was applied.
- That the authors are responsible for the content.

A usable template:

> Data processing scripts were drafted with the assistance of *[model, version]* accessed via *[tool, version]* in *[month year]*. All generated code was reviewed by the authors, executed locally, and validated against *[describe check: row counts, held-out subset, manual verification of n records]*. Prompts and model configuration are archived in the project repository at *[DOI/URL]*. No text in this manuscript was generated by a language model. The authors take full responsibility for the content.

Adjust the last two sentences to the truth. If a model was used for language editing, say so — most policies permit it and all of them require declaring it.

9.5 Containers, and their limits

A container pins the operating system, libraries and tool versions, which resolves the environment half of reproducibility. It does not resolve model non-determinism.

The strongest configuration available today:

```
# Pinned base, pinned model, pinned weights, no network at runtime
FROM docker.io/rocker/r-ver:4.4.1
COPY renv.lock .
RUN R -e 'renv::restore()'
COPY models/qwen3-coder-30b-q4.gguf /models/
# checksum verified at build:
#   sha256:4f2a...c91
```

This gives you a reproducible environment containing a reproducible model artefact, run offline. It is the closest thing to a deterministic AI-assisted pipeline currently achievable, and it is only possible with open weights.

Key point. *Reproducible AI-assisted research requires open-weight models running locally. An API-based workflow can be documented, but it cannot be reproduced, because the instrument is not under your control.*

Part IV

Local-First

10. The Local-First Approach

Kleppmann, Wiggins, et al. (2019) set out seven properties software should have if the user is to genuinely own their data: it should be fast, work across devices, work offline, support collaboration, last forever, be secure and private by default, and give the user ultimate ownership and control. The paper predates the current AI cycle and applies to it exactly.

10.1 The argument

10.1.1 Nothing leaves the machine

The decisive property for research. Unpublished occurrence records, sensitive locality data for threatened species, embargoed sequences, draft manuscripts, personal data from interviews or citizen science — none of it goes anywhere.

This is not hypothetical caution. Shared conversations from a commercial assistant were found surfacing in public search results containing API keys and personal data (International Business Times UK 2026). Data brokers routinely fail to comply with existing privacy law (Stanford Institute for Human-Centered Artificial Intelligence 2026b). A local model has no such failure mode, because there is no transmission.

For European institutions there is a compliance dimension. GDPR obligations around personal data, contractual restrictions in data-sharing agreements, and national rules on sensitive species localities are all substantially simpler when the answer to "where did the data go" is "nowhere".

10.1.2 No marginal cost

After the hardware, inference is free. This changes what you do. You stop rationing queries. You run the classifier over the entire corpus rather than a sample. You leave a batch job running overnight. Behaviour that is uneconomic at a per-token price is free once the machine is paid for.

10.1.3 It works offline

Research vessels. Field stations. Remote sites. Institutional networks with restrictive egress rules. Any of these makes a cloud-dependent workflow unusable, and all of them are ordinary conditions in ecology.

10.1.4 Digital sovereignty

The policy layer. European Commission, Joint Research Centre (2025) frames EU digital sovereignty as the capacity to exercise strategic independence in the digital domain while remaining open and connected — not autarky, but the ability to not be coerced. Anand et al. (2026) extract the concrete goals from EU legal texts. Meinhardt et al. (2026) survey

the commercial offerings that have appeared in response, and are sceptical: much of what is marketed as sovereign AI is a regional deployment of the same proprietary stack, which relocates the data without changing who controls the model.

Esposito et al. (2026) go further and propose a reference architecture — sovereignty as a design property of the system rather than a procurement checkbox applied afterwards. That is the right level at which to address it, though it presumes more freedom in system design than most institutions have.

For a public research institution the practical questions are narrower. Can you continue your work if a provider changes terms, raises prices, or withdraws from your market? Can you audit what you depend on? Are you building infrastructure that will outlive a commercial contract?

10.1.5 Longevity

An API endpoint is deprecated on the provider's schedule. A model file on your disk is not. Research infrastructure has a ten-year horizon; commercial API products frequently do not.

10.2 The costs, stated honestly

10.2.1 Capability

The gap is real. Frontier hosted models outperform anything you can run on a workstation on hard reasoning, long-context work, and multi-file code changes. For the most demanding tasks this gap matters and pretending otherwise wastes your time.

It is also narrowing, and the shape of the narrowing is favourable. Open-weight models in the 12–30B range now handle scoped coding, extraction, classification and summarisation well. Larger open-weight releases — dense models in the 27B class, mixture-of-experts models at much larger nominal parameter counts — close much of the remainder for users with the memory to run them.

10.2.2 Hardware and maintenance

Real money up front (Chapter 11) and real time in maintenance: drivers, runtimes, model management, debugging. Somebody has to do this. In a small group that somebody is a researcher not doing research, and that cost is frequently omitted from the comparison.

10.2.3 Speed

Local inference is slower, sometimes considerably. For interactive work this is noticeable. For batch work it is irrelevant — the job runs overnight either way.

Table 10.1: Deployment models and their tradeoffs.

Model	You get	You give up
SaaS (hosted chat)	Zero setup, best capability	Data control, versioning, offline use
API	Programmability, scale	Same, plus cost per token
MaaS (managed open weights)	Open models, no ops	Data still transits; vendor dependency
Self-hosted open weights	Full control, privacy, offline	Capability gap, hardware, ops burden

10.3 Open, closed, and the layers between

Model as a Service is the middle path that has emerged: vendors including Red Hat and Ollama offer managed or supported deployment of open-weight models. Brown (2026) argue for a hybrid foundation — workloads placed where their requirements put them rather than uniformly in one place. This is the pragmatic position and it is probably right for most institutions.

10.3.1 A sensible hybrid policy

- **Local, always:** unpublished data, sensitive localities, personal data, anything under an embargo or a restrictive data-sharing agreement.
- **Local, by default:** bulk processing, routine extraction, classification over large corpora, anything where per-token cost would otherwise constrain the design.
- **Hosted, acceptable:** public data, published literature, general explanation, hard reasoning tasks where the capability gap binds.
- **Hosted, with care:** code generation — check first whether your repository or its context is being transmitted, and whether your licence or institutional policy permits it.

Write this down as a group policy. The decision should not be made freshly by each person under time pressure.

10.4 A reference architecture

A layered design, arranged so that the parts which must last are the simplest ones.

Client
Browser on the workstation; mobile via the same web interface added to the home screen. No native applications required.

Interface
Open WebUI or LibreChat. One chat surface, model switchable per conversation. Owns no long-term knowledge.

Routing
An OpenAI-compatible API layer. Providers are interchangeable behind it — local Ollama,

or a hosted endpoint, chosen per request.

Runtime
Ollama or vLLM. Stateless. Input to output, no history, no opinions.

Retrieval
PostgreSQL with `pgvector` for embeddings, metadata and full-text search in one ACID store. LlamaIndex or Haystack for the pipeline.

Archive
Plain Markdown or text files. Append-only, human-readable, independent of any interface or database. This is the real long-term memory.

Sync and backup
File-based sync (Nextcloud, WebDAV, Dropbox, git), under normal 3-2-1 rules (Chapter 6).

The principle underneath the layering:

> *The UI is for talking, the models are for thinking, and plain text is for remembering.*

Interfaces are swappable. Models are stateless calculators. The Markdown archive is the notebook, and it is the only layer that must survive. Every component above it can be replaced without loss.

10.4.1 The logging proxy

One refinement worth the effort. Insert a small OpenAI-compatible proxy between the interface and the backends. It forwards requests unchanged and writes each exchange to a timestamped Markdown file with model name, role and content.

This gives you an append-only, greppable, audit-friendly trail that survives interface changes, database migrations and vendor departures. Organise it as `~/chatlogs/YYYY-MM/topic.md` and sync it with everything else. It is also, not incidentally, exactly the provenance record Chapter 9 asks for.

10.5 A FOSS stack, concretely

Permissively licensed throughout, sized for roughly a million documents:

- **Embeddings:** `bge-m3` (MIT) or `nomic-embed-text-v1.5` (Apache-2.0).
- **Vector store, metadata and full-text search:** PostgreSQL + `pgvector` (PostgreSQL licence), with PostGIS if the data is spatial.
- **Retrieval:** hybrid full-text plus vector, reranked with `bge-reranker`.
- **Generation:** an Apache-2.0 instruct model sized to your hardware — 12B if you have 32 GB, 30B if you have 64 GB.
- **Orchestration:** LlamaIndex or Haystack; LangGraph if the workflow is genuinely a graph.

- **Serving:** Ollama for single-user, vLLM for shared.
- **Interface:** Open WebUI.
- **Analytics:** DuckDB over Parquet for the tabular side.

Note what this stack does not require: an account, a contract, a network connection, or anyone's permission. For the operational details of standing one up — reverse proxy, TLS, model management, resource limits — confidence.sh (2026) is a practical walkthrough.

10.6 Decentralised ecology

Placitelli et al. (2026) is the proof that this is not theoretical for our field: a local-first, browser-based AI platform for sea turtle detection, built for decentralised ecological monitoring. Inference runs in the browser on the device that captured the image. No upload, no server, no connectivity requirement.

For citizen science this architecture is strictly better. Volunteers keep their data, the system works in the field where the connection does not, and there is no server cost that ends when the grant does.

Haas et al. (2025) generalise the programming model — convergent replicated dataflows that let developers write local operations while the runtime reconciles global state. Kleppmann, Frazee, et al. (2024) demonstrate the federated-identity version at social-network scale. The infrastructure for decentralised scientific applications is maturing, and biodiversity monitoring — distributed, intermittently connected, institutionally fragmented — is close to its ideal use case.

11. Hardware for Local Inference

Concrete numbers. They will age; the reasoning will not.

11.1 Why memory, not compute

Generating a token requires reading the model's weights through the compute units. Generate a thousand tokens and you have read the weights a thousand times. Token throughput is therefore governed by memory bandwidth, and the model has to fit in memory at all before bandwidth becomes the question.

This inverts the usual HPC intuition. A discrete GPU with enormous compute and 16 GB of VRAM cannot run a model that needs 40 GB — at any speed. A machine with 128 GB of moderately fast unified memory can.

Key point. *Capacity first: can the model fit? Bandwidth second: how fast can it be read? Compute third. Buying for FLOPs is the standard mistake.*

11.2 Unified memory architecture

Traditional PC architecture separates system RAM from GPU VRAM. Anything the GPU computes on must first be copied across PCIe, and VRAM is fixed at purchase.

Unified memory architectures — Apple Silicon, AMD's Ryzen AI Max, Qualcomm Snapdragon X, NVIDIA's Grace Blackwell desktop parts — put CPU and GPU on one memory pool. No copying, and capacity is whatever you specified. A laptop with 64 GB unified memory can allocate most of it to a model. A desktop with a 16 GB discrete GPU cannot, regardless of how much system RAM it has.

This is the reason Apple hardware is disproportionately represented in local inference work. It is an architectural advantage, not a brand preference, and AMD and Qualcomm are now shipping the same idea.

The trade is straightforward. A separate graphics card has far more bandwidth (1–3 TB/s against 200–800 GB/s) and is the only sensible choice if you also want to train models. A shared-memory machine holds far larger models (16–512 GB against 8–48 GB) and draws a fraction of the power — 30–120 W rather than 300–700 W.

If the model fits on the graphics card, the card is faster, often several times faster. The shared-memory advantage is that large models fit at all.

11.3 Machines, and what they run

Prices are indicative EUR at time of writing, for the memory configuration stated. They move.

Table 11.1: Representative machines for local inference.

Machine	Memory	Price	Comfortably runs	Power
Mac mini M4 Pro	24–64 GB	1,600–2,800	12–32B Q4	65 W
Mac Studio M4 Max/Ultra	64–192 GB	3,000–7,000	30–120B Q4	150 W
MacBook Pro M4 Max	36–128 GB	3,000–6,000	12–70B Q4	90 W
Framework Desktop (AI Max+ 395)	64–128 GB	2,000–2,800	30–70B Q4	120 W
Beelink GTR9 Pro / GMKtec EVO-X2	64–128 GB	1,800–2,500	30–70B Q4	120 W
NVIDIA DGX Spark (GB10)	128 GB	3,000–4,000	70–120B Q4	170 W
Workstation + RTX 5090	32 GB VRAM	3,500–5,000	12–32B Q4, fast	600 W

11.3.1 Reading the table

Under 2,000 EUR. A Mac mini M4 Pro at 48 GB, or an AMD Ryzen AI Max mini-PC at 64 GB. Both run a 30B model at Q4 usefully. This is the entry point for serious local work and it is a reasonable line item on most equipment budgets.

Around 3,000 EUR. A Mac Studio or a DGX Spark at 128 GB opens 70B models, which is where open-weight quality becomes genuinely competitive for scoped work. For a group, this is the sweet spot — one machine serving several people over the network via vLLM.

Discrete GPU. Choose this if you also need to train or fine-tune, or if you need maximum interactive speed on models under 32 GB. Note the power figure: 600 W continuous has implications for cooling, for the electricity bill, and for where in the building it can live.

11.4 Sizing by task

Table 11.2: What to buy for what you actually want to do.

Task	Model size	Memory	Notes
Text classification, NER	1–7B	8–16 GB	Often a fine-tuned small model beats a large general one
Summarising, extraction	7–14B	16–32 GB	Q4 is fine; quality loss is small
Coding, scoped edits	12–32B	32–64 GB	Needs 32K+ context; budget for the KV cache
Multi-file agentic work	30–70B	64–128 GB	Below this, expect frustration
Embeddings at scale	<1B	8 GB	CPU is adequate; this is throughput, not latency
Whisper transcription	1.5B (large-v3)	8–16 GB	Core ML or CUDA gives 3×

11.5 Quantisation

Quantisation reduces the numerical precision of the weights. Q4 — roughly four bits per parameter — cuts memory to about a quarter of FP16 with modest quality loss on most tasks.

Practical guidance: **Q4_K_M** is the default worth starting from. It is the best quality-per-byte point for most models and most tasks. **Q5** or **Q6** if you have headroom and the

task is precision-sensitive. **Q8** only if you have a specific reason. **Q2 and Q3** degrade noticeably; a smaller model at Q4 is usually better than a larger model at Q3.

Given a fixed amount of memory, a larger model at Q4 beats a smaller one at Q8. Size buys more than precision does, down to about four bits.

11.6 Models worth having locally

Names date quickly; the categories do not.

- **A coding model, 12–30B.** Dense Apache-2.0 models in this range handle scoped edits, explanation and boilerplate well. Verify tool-calling support before committing to one for agent use.
- **A general instruct model, 7–30B.** For summarisation, extraction and classification. Apache-2.0 licensing is available and worth insisting on.
- **An embedding model.** `bge-m3` or `nomic-embed-text`. Small, fast, and the foundation of any RAG pipeline.
- **A reranker.** `bge-reranker`. Cheap, and it improves retrieval quality more than most alternatives at the same cost.
- **Whisper large-v3.** For transcription.
- **A small fast model, 1–4B.** For classification at volume and for routing. Not everything needs the big model.

Check the licence on each. Several widely-used "open" models carry use restrictions that fail the Open Source Definition, and that matters if you are building infrastructure meant to be shared.

11.7 Shared deployment

For a group, one good machine beats several mediocre ones. Run vLLM on a server with 128 GB, expose an OpenAI-compatible endpoint on the institutional network, and point everyone's tooling at it.

Advantages: better hardware utilisation, one model set to maintain, centralised logging for provenance, no per-desk maintenance. The cost is a single point of failure and someone's time to administer it.

Listing 11.1: Minimal shared serving

```
# On the server
vllm serve Qwen/Qwen3-30B-A3B-Instruct \
    --host 0.0.0.0 --port 8000 \
    --max-model-len 32768

# On each workstation
export OPENAI_BASE_URL=http://gpu-server.local:8000/v1
export OPENAI_API_KEY=not-used-but-required
```

Put it behind the institutional firewall, not on a public address. An open inference endpoint is an open compute resource, and it will be found.

Part V

Risk, Integrity and Keeping Up

12. Security, Biorisk and Governance

Two distinct concerns share this chapter because they are entangled in practice: the security of systems you run, and the dual-use character of the biological capability these models provide. The entanglement is not incidental — the same safety mechanism frequently sits on both.

12.1 Running an agent is running arbitrary code

An agent harness executes shell commands on your machine, with your permissions, based on text generated by a model that can be influenced by any content it reads. Stated that plainly, the risk model is obvious.

Shapira et al. (2026) ran the experiment properly: autonomous agents in a live laboratory environment with persistent memory, email, Discord, file systems and shell access, probed by twenty researchers over two weeks under benign and adversarial conditions. Eleven representative failures are documented:

- Unauthorised compliance with instructions from non-owners.
- Disclosure of sensitive information.
- Execution of destructive system-level actions.
- Denial-of-service and uncontrolled resource consumption.
- Identity spoofing.
- Cross-agent propagation of unsafe practices.
- Partial system takeover.
- **Reporting task completion while the system state contradicted the report.**

The last one deserves its emphasis. It means an agent's own account of what it did is not evidence that it did it.

12.2 Prompt injection

The central unsolved problem. A language model does not reliably distinguish instructions from data. Any content it reads — a file, a web page, a README, a code comment, an issue on a repository, a CSV cell — can contain text that the model treats as a command.

This is not a bug with a fix pending. It follows from the architecture: both instructions and data arrive as tokens in the same context, and there is no channel separation. Mitigations reduce the rate; none eliminates it.

Attack surfaces that matter in research:

- A downloaded dataset with instructions in a metadata field.

- A PDF with invisible text (white on white, or off-page).
- A dependency whose README or docstrings carry injected text.
- A web page fetched during a literature search.
- An MCP server description — the tool's own self-description is untrusted input.

Key point. *Treat everything an agent reads as untrusted. This includes your own data files if they came from anywhere else, and it includes the descriptions that tools provide about themselves.*

12.3 Sandboxing

Defence in depth, with honest limits at each layer.

Permission prompts. The harness asks before running commands. Real protection, defeated by fatigue — after fifty prompts, people approve reflexively. Configure allowlists for genuinely safe read-only operations so the prompts you do see carry signal.

Filesystem restriction. Run the agent in a directory containing only what it needs. This is free and it is the single highest-value habit in this section.

Lightweight sandboxes. `bwrap` on Linux raises the bar. Escapes exist; treat it as a layer, not a boundary.

Containers. Podman or Docker isolate the namespace but share the host kernel. Kernel vulnerabilities have repeatedly produced container escapes and privilege escalation, so a container is not a security boundary against a determined attacker — though it is entirely adequate against an agent that misreads an instruction.

MicroVMs. A separate kernel, a genuine boundary. The correct answer for running agents with reduced supervision, and increasingly practical on Linux.

Network egress control. Underused and effective. An agent that cannot reach the network cannot exfiltrate, cannot fetch a malicious payload, and cannot be driven by remote content. If the task is local, cut the network.

Rashidi (2026) systematises 39 papers across 17 categories of execution security for coding agents — isolation, capability and access control, policy enforcement, time-of-check-to-time-of-use races, MCP threats, identity delegation, execution provenance, egress control, static analysis of generated code. The survey's own finding is that the literature is fragmented and rarely cross-cites, which is a reasonable proxy for the field's maturity.

The offensive side has developed in parallel. Qasemi (2026) classify AI-powered attacks and the defensive mechanisms available, and the summary for a research institution is unsurprising: phishing has become much harder to detect by writing quality, and reconnaissance against a target has become much cheaper. Neither is specific to AI research groups, but both raise the cost of a credential leak.

12.4 Supply chain

Slopsquatting. Models hallucinate package names. Attackers register the hallucinated names. The generated `install.packages()` call then installs an attacker's package. Verify every dependency an agent adds exists and is the one you meant — this is the direct, practical reason for the "do not add dependencies without asking" line in Chapter 7.

Model artefacts. Downloading weights is executing code from an untrusted source if the format permits it. Pickle-based formats can carry arbitrary payloads. Prefer safetensors, verify checksums, and note that this has already produced real incidents (OpenAI 2026b).

MCP servers. An MCP server is a program you run with access to your data, and its tool descriptions enter the model's context as trusted-looking text. Apply the SNIFF criteria (Nagraj et al. 2026) before installing one: who maintains it, is the project active, does it fit, how fragile are its dependencies.

12.5 A practical baseline

1. Never run an agent with elevated privileges. There is no task that requires it.
2. Run it in a project directory, not in your home directory.
3. Version control everything, commit before, review the diff after.
4. Keep credentials out of the working tree and out of the environment the agent can read.
5. Rotate any key that has ever been in a context window.
6. Disable network egress for tasks that do not need it.
7. Use a container for anything running with reduced supervision; a microVM if it is running unattended.
8. Read every dependency addition.
9. Never point an agent at production infrastructure.

Key point. *Credentials in a chat context should be considered compromised. Shared conversations have surfaced in public search results with API keys intact (International Business Times UK 2026). Rotate, do not rationalise.*

12.6 Dual use in biology

The biological capability of frontier models is a live policy question, and the evidence is stronger than it was.

Brent and McKelvey (2025) present dialogues in which deployed foundation models provide guidance on elements of technically demanding development tasks, including construction of viral pathogens, and argue this constitutes a real increase in biological weapons risk. C. B. C. Zhang et al. (2026) ran the controlled version: a multi-model, multi-benchmark human uplift study comparing novices with LLM access against internet-only access across

eight biosecurity-relevant task sets, with up to thirteen hours per task. They found substantial uplift.

The counterweight is calibration. Peppin et al. (2025) argue that much biorisk discourse outruns the evidence, and that the operative constraints on biological weapons development are tacit knowledge, materials access and laboratory skill rather than information availability. Turner (2026) examines the benchmark evidence specifically: frontier models now match or exceed expert reference points on many biosecurity-relevant evaluations and several benchmarks are approaching saturation, but these results have not been calibrated against physical task performance. Benchmark scores are highly sensitive to format.

Both can be true. Information barriers have fallen; physical barriers have not. The question is how much of the total barrier was informational, and it is not settled.

12.6.1 The correlation with cybersecurity

The mechanism linking the two halves of this chapter.

A model's refusal behaviour — declining to assist with pathogen engineering — is implemented in the instruction-tuning layer, in the same place and by the same mechanism as its refusal to write malware. Prompt injection defeats both, because injection subverts instruction-following in general rather than any particular refusal.

The consequence, stated by practitioners and worth repeating: *prompt injection defences open a biosecurity gap.* Robust separation of instructions from data would strengthen both simultaneously. Its absence weakens both simultaneously. They are not independent problems.

Holko (2026) draws the structural conclusion. Biosecurity frameworks are organised around control of known agents, materials and individual technologies. In AI-enabled biology, capability comes from combination — from the way data, models, infrastructure and workflows fit together. The risk is not in any one part but in how they connect. Nucleic acid synthesis screening, effective in isolation, may not capture risks that emerge from an integrated system.

Barkan et al. (2026) examine the natural response — software barriers restricting AI agent use of biological design tools — and are measured about feasibility. Restriction at the tool layer is more tractable than restriction at the model layer, because tools have identifiable interfaces.

For working ecologists, none of this changes daily practice. It does explain why some models decline biological questions that seem benign, why open-weight release decisions are contested, and why sequence data governance is becoming stricter. It is worth understanding the argument before being asked for an opinion on it.

12.7 Governance and risk taxonomy

Slattery et al. (2026) assembled 1,725 risks from 74 frameworks into a structured database with two taxonomies: causal (who caused it, was it intentional, before or after deployment) and domain (seven areas from discrimination and privacy through misinformation to weapons). Its value is shared vocabulary — the paper's own motivating observation is

that the same word describes different problems across communities while different words describe identical concerns.

For an institutional AI policy, this is a better starting point than drafting from scratch.

12.7.1 What an institutional policy should cover

- **Data classification.** What may go to a hosted service, what may not. Be specific: sensitive localities, personal data, embargoed sequences, third-party data under agreement.
- **Approved tools.** A short list with a named owner, rather than ad-hoc adoption.
- **Disclosure.** What must be declared in publications, and where.
- **Agent execution.** Sandboxing requirements, and an explicit prohibition on production access.
- **Credentials.** Rotation policy after context exposure.
- **Procurement.** Whether a service that trains on submitted data is acceptable, and under what contract terms.

12.8 The wider frame

Two strands in the literature situate this in longer time.

Müller et al. (2026) consider AI systems whose components, learning rules and deployment conditions can themselves undergo Darwinian evolution, and ask under what technical and ecological conditions this becomes possible. They argue the scenario is underappreciated in existential risk debates. Whether or not one credits the conclusion, the analytical move — treating deployed AI systems as an evolving population under selection — is one ecologists are unusually well equipped to evaluate.

Korecki (2025) makes the complementary argument about values. AI ethics is overwhelmingly anthropocentric; recent extensions reach sentientism; neither captures the biosphere. He proposes biospheric ethics as a design target and asks what an AI aligned with biospheric rather than human interests would look like.

These connect to an older line of thought — Vernadsky's transition from biosphere to noosphere (Vernadsky 1938), revisited by Vidal (2024) as planetary superorganism and major evolutionary transition, and pushed further by Lovelock and Appleyard (2019), who argued that the Anthropocene is ending and a Novacene beginning in which new beings emerge from existing AI systems. Neumann (1955) asked the version of the question that has aged best, in 1955, about a different technology: whether the environment in which technological progress occurs has become too small and too disorganised for what is being built in it.

These are not operational guidance. They are the background against which the operational questions are being asked, and ecologists have a professional stake in them that most participants in the debate do not.

13. Changes Across the Scientific Workflow

Where these tools actually land in the work, stage by stage, with the verification each stage requires.

13.1 Literature retrieval

The stage that has changed most.

Keyword search over an exponentially growing literature has been failing for a while. W. Zhang et al. (2025) trace the progression from keyword search through retrieval-augmented generation to *agentic deep research*: systems that decompose a question, search iteratively, read what they find, refine the query based on it, and synthesise across sources. The difference from RAG is the loop — the system decides what to search for next based on what it has read.

For a genuinely multi-part question — "what is known about thermal tolerance in Mediterranean seagrass, including grey literature and non-English sources" — this outperforms manual search substantially, both in recall and in time.

13.1.1 Resources

Table 13.1: Literature sources, and what each is good for.

Resource	Use
PubMed	Biomedical and molecular; excellent structured metadata
OpenAlex	Open bibliographic graph; free API; good for bulk analysis
Google Scholar	Broadest coverage including grey literature; no usable API
Biodiversity Heritage Library	Historical taxonomic literature; irreplaceable
Biodiversity PMC	Biodiversity-specific full text (Pasche et al. 2026)
Web of Science, Scopus	Curated, subscription; citation analysis
Dimensions	Links publications to grants, patents, policy documents
Zenodo / BioSysLit	Preprints, datasets, community collections

OpenAlex deserves particular mention for this work: a free API over the full bibliographic graph makes it the natural backend for a custom retrieval pipeline, and it does not impose the query limits that make Scholar unusable programmatically.

13.1.2 The verification requirement

Every reference gets resolved. Zhao et al. (2026) found roughly 147,000 hallucinated citations in 2025 alone across the major preprint and literature servers. A hallucinated reference has real authors, a real journal and a plausible title; it fails only on lookup.

Listing 13.1: Resolve every DOI before it enters a manuscript

```
# Extract DOIs from a .bib and check each resolves
```

```
grep -oP 'doi\s*=\s*\{\K[^}]+' references.bib | while read -r doi; do
    code=$(curl -s -o /dev/null -w "%{http_code}" \
           -H "Accept: application/json" "https://doi.org/$doi")
    [ "$code" = "200" ] || echo "UNRESOLVED: $doi"
done
```

Entries without a DOI need manual checking. Do it.

Key point. *Never cite a paper you have not opened. An AI-assisted search is a way to find candidates, not a way to avoid reading them.*

13.2 Reading

Models are good at explaining a paper's method, at translating, and at answering specific questions from a document you supply. That last qualifier matters: with the PDF in context, the model is constrained by it. Asked about a paper from memory, it will invent details.

If you are building a corpus, the quality of the PDF extraction sets the ceiling for everything after it. Auer et al. (2024) describe Docling — MIT-licensed, layout-aware, with table structure recognition, running on commodity hardware. Naive text extraction mangles multi-column layouts, drops tables, and interleaves figure captions into body text; everything downstream inherits that damage.

What models cannot do is judge whether a paper is any good. Sample size adequacy, whether controls were appropriate, whether the statistics match the design, whether the conclusion follows — this is domain judgement, and the confident summary of a bad paper reads exactly like the confident summary of a good one.

13.3 Analysis

Covered in Chapter 7 and Chapter 9. The summary: use models to generate code, not results. Code is inspectable, deterministic, version-controllable and reviewable. A number produced directly by a model is none of those.

The verification points specific to ecological data, repeated because they are where real errors occur:

- Row counts before and after every join.
- Coordinate reference systems — explicit, never inferred.
- Taxonomic names matched against a backbone before joining, never joined as raw strings.
- Units, including the ones embedded in column names.
- Date parsing, especially with mixed formats and two-digit years.
- Missing data treatment — silently dropped rows are the most common serious error in this domain.

13.4 Writing

The most contested stage, with a literature that has converged faster than expected.

Salvagno et al. (2023) made the early case: these tools help with organisation, first drafts and proofreading, and the responsibility for content remains with the authors. Granjeiro et al. (2025) survey the tools and ethical implications more broadly.

The interesting strand is authorial voice. Al Hosni (2025) reviews the evidence that generative tools homogenise academic prose — the boundary between author-written and AI-generated text blurring in a way that costs something real. Singh et al. (2025) take the constructive approach, describing a system designed to *develop* authorial voice rather than substitute for it, supporting both writing performance and authorship belief.

The distinction is worth holding onto. A tool that drafts for you and a tool that helps you draft are different tools, and only one of them leaves you a better writer.

13.4.1 Policy

Nature Methods Editorial (2026) sets out where journal policy has settled, and the convergence across publishers is substantial:

- **AI cannot be an author.** Authorship entails accountability, which requires an entity that can be held accountable.
- **Use must be disclosed.** In methods or acknowledgements, specifically.
- **Authors are fully responsible** for all content, including anything a model produced.
- **Peer review is generally off-limits.** Uploading a confidential manuscript to a third-party service breaches confidentiality. Some publishers prohibit it outright.

That last point is frequently overlooked and is the most serious. A manuscript under review is confidential. Pasting it into a hosted chat interface transmits it to a third party, may enter a training corpus, and is a breach of the reviewer's undertaking. If you want model assistance with a review, use a local model. This is one of the cleanest cases for local-first in this book.

13.5 Peer review and editorial work

Automated screening — reference resolution, statistical reporting checks, figure duplication detection, methods completeness — is legitimate and useful, and does not require a generative model.

Automated review judgement is not. The reviewer's contribution is domain judgement about whether the work is sound and whether the conclusions follow. A model produces text shaped like a review without the judgement, and its plausibility is the problem rather than the value.

13.6 Synthesis and assessment

Krug and Ruch (2026) examine AI in IPBES assessments and pose the hype-or- salvation question without answering it, which is appropriate. Assessment work has a formal requirement that every statement be traceable to a source and carry a calibrated confidence term. Generative summarisation breaks the first and fabricates the second.

Where AI helps in assessment: finding candidate literature, extracting structured data from many papers into a consistent schema, translating non-English sources, flagging inconsistencies. Where it does not: writing the assessment.

13.7 Detection and its misuse

Institutions want to detect AI-written text. The tools are not reliable enough to support decisions about people. Pudasaini et al. (2026) show that detectors reporting high benchmark accuracy largely exploit dataset artefacts, with accuracy collapsing out of distribution — and a systematic bias against non-native English writing, which in ecology means a systematic bias against a large fraction of the field.

The answer is disclosure policy, not detection technology. Make the expected declaration easy and normal, and the incentive to conceal drops.

13.8 A summary table

Table 13.2: Stage by stage: what helps, and what must be checked.

Stage	**Genuinely useful for**	**What you must check**
Literature search	Query expansion, multi-source synthesis, translation	Resolve every DOI; read what you cite
Reading	Method explanation, targeted Q&A with PDF in context	Quality judgement stays yours
Data extraction	Unstructured text to structured records	Manual validation of a random subset
Analysis	Code generation, debugging, plotting	Run it; check joins, units, CRS, row counts
Writing	Structure, first draft, language editing	Every claim; every citation; your voice
Review	Nothing involving the manuscript on a hosted service	Use a local model or none
Assessment	Candidate finding, structured extraction	Traceability to source; confidence terms

14. How to Keep Up

You cannot. That is the correct starting position, and accepting it is more useful than any reading list.

14.1 Why it is impossible, structurally

The field publishes faster than it can be read. Model releases arrive weekly, benchmarks are superseded within months, and the tooling layer turns over faster than documentation can follow. Nobody is across all of it, including the people who appear to be.

The practical response is to be deliberate about what you track and to accept gaps everywhere else.

14.2 What is actually stable

Sort what you could learn by how long it stays true. This is the most useful filter there is.

Table 14.1: What to learn, and how long it lasts.

Layer	Stays true for	Worth learning?
Specific model names and versions	Weeks	No. Look them up.
Benchmark scores	Months	No. They do not transfer to your task.
Pricing and rate limits	Months	Only when procuring.
Tool names and CLI flags	6–12 months	Lightly; read the docs when needed.
Architectural patterns (harness, RAG, agent loop)	Years	Yes.
Failure modes (hallucination, sycophancy, injection)	Years	Yes, thoroughly.
Data standards, provenance, reproducibility	Decades	Yes. This is the part that lasts.
Domain expertise in ecology	Career	Yes. It is what makes you useful here.

Key point. *Invest your learning time at the bottom of that table. Everything at the top is a lookup, and treating it as knowledge to be maintained is how the field consumes attention without returning anything.*

14.3 People worth following

A short list beats a long one. These are people whose output is worth reading because they are careful, not because they are fast.

Melanie Mitchell (Santa Fe Institute)
The most reliable sceptical voice on AI capability claims. Mitchell (2026) and her work with Krakauer et al. (2026) are the antidote to both hype and dismissal.

Christopher Manning (Stanford)
NLP foundations; co-author on the legal RAG evaluation (Magesh et al. 2025) that is the best empirical work on retrieval grounding's limits.

Lars Juhl Jensen (Copenhagen)
Biomedical text mining, dictionary-based NER, and practical tooling. The pre-LLM text mining tradition he represents is still the right approach for a great many problems.

Sophia Ananiadou (Manchester)
Biomedical NLP, in the department where Turing worked — a lineage from cryptanalysis to text mining that is more than a curiosity.

Anne Thessen
Biodiversity informatics, ontologies, and the data layer this all depends on.

Martin Kleppmann (Cambridge)
Local-first software (Kleppmann, Wiggins, et al. 2019) and distributed systems. The architectural case for owning your data.

Hadley Wickham (Posit)
On what an agent is, and on AI in data analysis, from someone who has thought hard about tool design for scientists.

14.4 Sources, by type

14.4.1 Primary

arXiv `cs.CL`, `cs.AI`, `cs.IR`; bioRxiv bioinformatics. Do not attempt to follow these; search them when you have a question. Set alerts on narrow terms only.

For the field-level picture once a year, the AI Index (Stanford Institute for Human-Centered Artificial Intelligence 2026a) is the best single document: comprehensive, data-driven, and explicit about its methodology.

14.4.2 Curated lists

Awesome-lists on GitHub for RAG, agentic deep research, and AI agents remain a reasonable way to discover tooling. Check the last commit date — a stale awesome-list is worse than none.

14.4.3 Distribution and documentation

Hugging Face for models and datasets. Ollama's library for what runs locally. The documentation for whichever harness you use, which changes faster than any third-party guide can track — read the source documentation, not a blog post about it.

14.4.4 Community

Your domain community is more useful than the AI community for your purposes. It is also worth knowing its actual composition: national surveys of computational capacity

in environmental omics (Theodosopoulou 2022) tend to find that what actually holds groups back is training and keeping staff, not tooling, which is a useful corrective when the discussion turns to hardware. TDWG for biodiversity standards. ELIXIR training and TeSS for life science computing. GBIF and OBIS community channels. The World Biodiversity Forum, where Pasche et al. (2026) and Krug and Ruch (2026) both appeared — the people working on AI *for biodiversity* specifically are a small and findable group.

14.5 A sustainable routine

Weekly, 30 minutes
Skim one aggregator. Note things to look up later. Do not follow links.

Monthly, 2 hours
Try one thing. A model, a tool, a technique, on a real task from your own work. Hands-on beats reading by a wide margin, and a real task beats a tutorial.

Quarterly, half a day
Review your stack. Is there something you are doing manually that should be automated, or automated that should be manual? Update pinned versions. Test a backup restore.

Annually, a day
Read the AI Index. Revisit your group's policy on data classification and disclosure. Reassess hardware.

That is roughly forty hours a year. More than that starts competing with doing ecology.

14.6 Learning by building

The most efficient path is a small real project. Pick something you actually need:

- A RAG system over your own reprint collection.
- A classifier for your camera trap images.
- An extraction pipeline for a corpus of species descriptions.
- An agent-assisted refactor of an analysis you have been avoiding.

Any one of these teaches more than a year of reading, because the failures are specific and yours. You will discover that PDF extraction is the hard part, that chunking strategy matters more than model choice, that the model is confidently wrong in a way you can now recognise. None of that transfers from a blog post.

14.7 What to tell colleagues who ask

You will be asked. A usable short answer:

> These systems are genuinely useful for transforming text and generating code, where you supply the content or can cheaply check the output. They are unreliable for anything requiring verified fact — they fabricate citations at

> measurable rates and agree with you when you are wrong. The main change to daily practice is agentic coding, which is more a system administration skill than a programming one. The main thing to get right is data organisation and provenance, and that was worth getting right anyway. Run models locally where you can, for privacy, cost and reproducibility. Verify everything that matters, and disclose what you used.

14.8 A closing note

The pressure to adopt is institutional, and it is not always aligned with doing good work. It is legitimate to conclude that a particular AI method is not appropriate for a particular problem, and Mishra (2026) makes the general version of that case for ecology seriously and well.

It is equally legitimate to use these tools heavily and with discipline. What is not reasonable is using them without checking, or staying out of the argument while other people make decisions about your data, your infrastructure and your field.

The skills that last here are the ones ecology already values: knowing where data came from, being sceptical of a result that is too convenient, documenting what you did so someone else can check it. Those transfer. Everything else is a lookup.

Part VI

Appendices

A. Glossary

Agent
A language model given tools and a loop, able to take actions rather than only produce text.

Agent harness
The orchestration system around an agent: context assembly, tool execution, permissions, memory, error recovery. Where the real engineering differences between systems live.

AGENTS.md / CLAUDE.md
A plain-text file in a project stating conventions and constraints for agents working in it. A behavioural contract, written in response to observed failures.

Attention
The transformer operation letting each position in a sequence attend to every other position in parallel. The architectural change that made training at scale practical.

Chunking
Splitting documents into passages for embedding and retrieval. Chunk size and boundary strategy affect RAG quality more than model choice.

CHOP (Chat-Oriented Programming)
Iterative development in dialogue with a model, with review at each step. Distinguished from *vibe coding* by the presence of verification.

Context window
The maximum tokens a model can attend to in one call. Everything it knows during that call is in the window. Attention degrades toward the middle of long contexts.

Embedding
A dense vector representation of text, such that semantically similar text maps to nearby vectors. The basis of vector retrieval.

FAIR
Findable, Accessible, Interoperable, Reusable. The data management principles biodiversity informatics has been implementing since well before "AI-ready" existed.

Fine-tuning
Further training of a pretrained model on domain data. Distinct from RAG, which supplies knowledge at inference time without changing weights.

Hallucination
Fluent, plausible, specific and false output. *Confabulation* is the more accurate term. Structural rather than incidental: training rewards guessing over abstention (Kalai et al. 2025).

Inference
Running a trained model to produce output. What you do locally; training is what you do not.

Jagged intelligence
Mitchell (2026)'s term for capability that is uneven and unpredictable — strong on hard tasks, failing on easy ones, with failures not inferable from successes.

KV cache
Cached key and value tensors from previous tokens, avoiding recomputation during generation. Consumes memory proportional to context length; budget for it when sizing hardware.

LLM
Large Language Model. A function mapping a token sequence to a probability distribution over the next token. Stateless.

Local-first
Software architecture in which data lives on the user's device and works without a network, with sync as an enhancement rather than a requirement (Kleppmann, Wiggins, et al. 2019).

MaaS (Model as a Service)
Managed deployment of open-weight models by a vendor. A middle path between SaaS and self-hosting.

MCP (Model Context Protocol)
A standard for exposing tools and data sources to language models. Wrap a resource once rather than once per client. Server descriptions enter the model's context and are untrusted input.

Open weight
Model parameters are downloadable. Distinct from open source, which would additionally require training data, code and pipeline (Stanford Institute for Human-Centered Artificial Intelligence 2026c).

Prompt injection
Instructions embedded in content a model reads, which the model executes as commands. Unsolved, because instructions and data share one channel with no separation.

Provenance
Documented origin and processing history of data or a result. Absent by default in generative output; must be added by logging.

Quantisation
Reducing weight precision to save memory. Q4 is roughly four bits per parameter, about a quarter of FP16, with modest quality loss.

RAG (Retrieval-Augmented Generation)
Retrieving relevant documents and supplying them in the prompt, giving a model access to knowledge it was not trained on. Reduces but does not eliminate hallucination (Kurniawan et al. 2024).

Reranking
Re-scoring retrieved candidates with a more expensive cross-encoder. A cheap and effective quality improvement in RAG pipelines.

Slopsquatting
Registering package names that models hallucinate, so that generated install commands

fetch attacker-controlled code.

Sycophancy
Excessive agreement with and validation of the user. Measurably reduces prosocial intent and increases dependence (Cheng et al. 2026); can drive even rational updating toward false beliefs (Chandra et al. 2026).

Token
A subword chunk of text. Roughly four characters or three-quarters of a word in English. The unit of both context and billing.

Transformer
The neural network architecture underlying current language models, introduced 2017. Replaced sequential recurrence with parallel attention.

Unified memory
Architecture in which CPU and GPU share one memory pool. Enables running large models on consumer hardware, since capacity is not limited to a fixed VRAM allocation.

Vibe coding
Accepting AI-generated code without close review. Acceptable for throwaway work; not for anything entering a manuscript, database or shared pipeline.

B. Checklists

Detachable. Print them if that helps.

B.1 Before starting an AI-assisted project

- ☐ Data classified: what may go to a hosted service, what may not.
- ☐ `README.md` written — what, why, how to run, current state.
- ☐ `AGENTS.md` written — conventions and explicit prohibitions.
- ☐ Git initialised; `.gitignore` excludes data and credentials.
- ☐ `data/raw/` populated, set read-only, checksummed.
- ☐ Environment pinned (`renv.lock`, `uv.lock`).
- ☐ Backup verified: three copies, two media, one off-site.
- ☐ Decided which model, hosted or local, and why.

B.2 Before each agent session

- ☐ Working tree clean, or checkpoint committed.
- ☐ Agent started in the project directory, not `$HOME`.
- ☐ No credentials in the working tree or the readable environment.
- ☐ Task scoped to something reviewable in one sitting.
- ☐ Fresh session if the task changed.

B.3 After each agent session

- ☐ `git diff` read in full. All of it.
- ☐ Any new dependency verified to exist and to be the intended package.
- ☐ Code executed; output inspected, not just the exit status.
- ☐ Row counts checked before and after every join.
- ☐ Edge cases exercised: empty input, single row, `NA`s, duplicates.
- ☐ Run twice; output identical.
- ☐ Result sanity-checked against domain expectation.
- ☐ Prompt, model, version and parameters logged to `logs/`.
- ☐ Committed with a message stating what changed and why.

B.4 Before citing anything

- ☐ Every DOI resolves.
- ☐ Every reference without a DOI verified manually.
- ☐ Every cited paper actually opened and read.
- ☐ Author lists, years and journals match the real record.
- ☐ The claim you attribute is the claim the paper makes.

B.5 Before submitting a manuscript

- ☐ Every reference resolved (again — references get added late).
- ☐ AI use disclosed in methods or acknowledgements, specifically.
- ☐ Model, version and date recorded.
- ☐ Verification procedure described.
- ☐ Code and prompts archived with a DOI (Zenodo).
- ☐ Data deposited in the appropriate repository.
- ☐ No confidential material was sent to a hosted service.
- ☐ Every author has read the final text and can defend every claim.

B.6 Before reviewing a manuscript

- ☐ Checked the journal's policy on AI in peer review.
- ☐ Manuscript NOT uploaded to any hosted service. Local model or none.
- ☐ Judgement about soundness is yours, not a model's.

B.7 Evaluating a tool before adopting it (SNIFF)

After Nagraj et al. (2026):

- ☐ **Source** — Who maintains it, and why? Funded, or a side project?
- ☐ **Network** — Part of a wider ecosystem, or isolated?
- ☐ **Interaction** — Are developers and users active? Issues answered?
- ☐ **Fit** — Does it match your use case and your licence requirements?
- ☐ **Fragility** — How many dependencies? How current? Security practices?

B.8 Sizing hardware for local inference

- ☐ Largest model you need identified.
- ☐ Memory computed: parameters × 0.6 for Q4, plus 25% overhead.
- ☐ Context window requirement added to the memory budget (KV cache).
- ☐ Unified memory considered against discrete GPU for that capacity.
- ☐ Power and cooling checked against where the machine will live.
- ☐ Shared deployment considered — one good machine usually beats several.

B.9 Institutional AI policy: minimum contents

- ☐ Data classification rules, with specifics, not principles.
- ☐ Approved tools, each with a named owner.
- ☐ Publication disclosure requirements.
- ☐ Peer review prohibition on hosted services.
- ☐ Agent sandboxing requirements; no production access.
- ☐ Credential rotation after context exposure.
- ☐ Procurement terms: does the vendor train on submitted data?
- ☐ Review date. Annually is right.

References

Al Hosni, J. (2025). "Preserving Authorial Voice in Academic Texts in the Age of Generative AI: A Thematic Literature Review". In: *Arab World English Journal* 16.3, pp. 244–258. DOI: 10.24093/awej/vol16no3.14.

Anand, S., S. Donnelly, M. Ehrenhard, L. J. M. Nieuwenhuis, and A. Abhishta (2026). "Digital sovereignty and the means to European digital security: identification of the goals related to digital sovereignty from legal texts published by the European Union". In: *European Security*. DOI: 10.1080/09662839.2026.2627905.

Anna's Archive (2026). *AI companies destroy physical books — let's scan rare books before it's too late*. Anna's Blog. URL: https://annas-archive.gl/blog/physical-destruction.html.

Anthropic (2026). *The Complete Guide to Building Skills for Claude*. Tech. rep. Anthropic.

Auer, C., M. Lysak, A. Nassar, M. Dolfi, N. Livathinos, and P. W. J. Staar (2024). "Docling Technical Report". In: arXiv: 2408.09869 [cs.CL].

Barkan, C. O., C. Rodriguez, S. Chowdhury, L. A. Zhang, and R. Brent (2026). *Restricting AI Agent Use of Biological Tools: Exploring the Feasibility of Software Barriers*. Tech. rep. RR-A5000-1. RAND Corporation.

Brent, R. and T. G. McKelvey (2025). *Contemporary AI foundation models increase biological weapons risk*. Tech. rep. WR-A3853-1. RAND Global and Emerging Risks.

Brixi, G., M. G. Durrant, J. Ku, et al. (2026). "Genome modelling and design across all domains of life with Evo 2". In: *Nature*. DOI: 10.1038/s41586-026-10176-5.

Brown, A. (2026). *The future of AI demands a hybrid foundation*. Red Hat Blog. URL: https://www.redhat.com/en/blog/future-ai-demands-hybrid-foundation.

Castro, A., J. Pinto, L. Reino, P. Pipek, and C. Capinha (2024). "Large language models overcome the challenges of unstructured text data in ecology". In: *Ecological Informatics* 82, p. 102742. DOI: 10.1016/j.ecoinf.2024.102742.

Chandra, K., M. Kleiman-Weiner, J. Ragan-Kelley, and J. B. Tenenbaum (2026). "Sycophantic Chatbots Cause Delusional Spiraling, Even in Ideal Bayesians". In: arXiv: 2602.19141 [cs.AI].

Cheng, M., C. Lee, P. Khadpe, S. Yu, D. Han, and D. Jurafsky (2026). "Sycophantic AI decreases prosocial intentions and promotes dependence". In: *Science*. DOI: 10.1126/science.aec8352.

confidence.sh (2026). *Local AI Stack: How To Selfhost Your Own AI Inference Infra*. confidence.sh blog. URL: https://confidence.sh/blog/how-to-selfhost-your-own-ai-inference-infra/.

Coppola, G. (2026). "Transformers are Bayesian Networks". In: arXiv: 2603.17063 [cs.AI].

Du, P. (2026). "Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers". In: arXiv: 2603.07670 [cs.AI].

Esposito, M., L. Marchesi, R. Tonelli, and V. Lenarduzzi (2026). "Sovereign-by-Design: A Reference Architecture for AI and Blockchain Enabled Systems". In: arXiv: 2602.05486 [cs.SE].

European Commission, Joint Research Centre (2025). *Open but Not Powerless: Towards a Common Understanding of EU Digital Sovereignty*. Tech. rep. JRC144908. JRC Science for Policy Brief, Digital Sovereignty Series.

Feng, T., T. H. Trinh, G. Bingham, et al. (2026). "Towards Autonomous Mathematics Research". In: arXiv: 2602.10177 [cs.LG].

Gill, I. S., E. J. Griffiths, D. Dooley, et al. (2023). "The DataHarmonizer: a tool for faster data harmonization, validation, aggregation and analysis of pathogen genomics contextual information". In: *Microbial Genomics* 9. DOI: 10.1099/mgen.0.000908.

Granjeiro, J. M., A. A. Del Bel Cury, J. A. Cury, M. Bueno, M. D. Sousa-Neto, and C. Estrela (2025). "The Future of Scientific Writing: AI Tools, Benefits, and Ethical Implications". In: *Brazilian Dental Journal* 36, e25–6471. DOI: 10.1590/0103-644020256471.

Guo, W., M. Zhang, B. Han, et al. (2026). "PromptBio-Bench: Benchmarking LLM-based Bioinformatics Agents for End-to-End Data Analysis". In: *bioRxiv*. DOI: 10.64898/2026.05.05.723092.

Haas, J., C. Kuessner, R. Mogk, and M. Mezini (2025). "Think Locally, Act Globally: A Programming Model for Decentralized Applications". In: *IEEE Internet Computing*. DOI: 10.1109/MIC.2025.3618947.

Holko, M. (2026). "Toward relational biosecurity: understanding AI-enabled biology as a connected system". In: *Frontiers in Microbiology* 17, p. 1856819. DOI: 10.3389/fmicb.2026.1856819.

Huang, K., S. Zhang, H. Wang, et al. (2025). "Biomni: A General-Purpose Biomedical AI Agent". In: *bioRxiv*. DOI: 10.1101/2025.05.30.656746.

International Business Times UK (2026). *Claude Shared Chats Surface in Search Results Containing API Keys and Personal Data*. IBTimes UK.

Ioannou, A., E. Bataka, N. Kokosis, C. Billinis, and C. Laspidou (2025). "One Health in Coastal and Marine Contexts: A Critical Bibliometric Analysis Across Environmental, Animal, and Human Health Dimensions". In: *International Journal of Environmental Research and Public Health* 22.10, p. 1523. DOI: 10.3390/ijerph22101523.

James, K. (2026). *Why is Anthropic destroying books?* The Guardian. URL: https://www.theguardian.com/commentisfree/2026/aug/05.

Jiang, J., Y. Li, S. Cao, et al. (2025). "Artificial intelligence in bioinformatics: a survey". In: *Briefings in Bioinformatics* 26.6, bbaf576. DOI: 10.1093/bib/bbaf576.

Johnson, C. A., E. M. Willigan, E. Y. Hwang, et al. (2026). "Unlocking biodiversity data with robotic imaging and AI-driven transcription of natural history collections". In: *Science Advances*. DOI: 10.1126/sciadv.aeb6975.

Kalai, A. T., O. Nachum, S. S. Vempala, and E. Zhang (2025). "Why Language Models Hallucinate". In: arXiv: 2509.04664 [cs.CL].

Kimi Team (2026). *Attention Residuals*. Tech. rep. arXiv:2603.15031. Moonshot AI. URL: https://github.com/MoonshotAI/Attention-Residuals.

Kleppmann, M., P. Frazee, J. Gold, and J. Graber (2024). "Bluesky and the AT Protocol: Usable Decentralized Social Media". In: *Proceedings of the ACM CoNEXT 2024 Workshop*. DOI: 10.1145/3694809.3700740.

Kleppmann, M., A. Wiggins, P. van Hardenberg, and M. McGranaghan (2019). "Local-First Software: You Own Your Data, in spite of the Cloud". In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)* DOI: 10.1145/3359591.3359737.

Kõljalg, U., K. Abarenkov, A. Zirk, et al. (2019). "PlutoF: Biodiversity data management platform for the complete data lifecycle". In: *Biodiversity Information Science and Standards* 3, e37398. DOI: 10.3897/biss.3.37398.

Korecki, M. (2025). "Biospheric AI". In: arXiv: 2401.17805 [cs.CY].

Krakauer, D. C., M. Mitchell, and J. W. Krakauer (2026). "Large language models and emergence: a complex systems perspective". In: *Philosophical Transactions of the Royal Society A* 384.2320, p. 20250014. DOI: 10.1098/rsta.2025.0014.

Krug, R. and P. Ruch (2026). "The Role of AI in IPBES Assessments and Biodiversity Literature Analysis — Hype or Salvation?" In: *World Biodiversity Forum 2026*. DOI: 10.5194/wbf2026-968.

Kurniawan, S., S. Thudumu, and S. Barnett (2024). "Seven Failure Points When Engineering a Retrieval Augmented Generation System". In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN)*. DOI: 10.1145/3644815.3644945.

Liu, J., X. Zhao, X. Shang, and Z. Shen (2026). "Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems". In: arXiv: 2604.14228 [cs.SE].

Lovelock, J. and B. Appleyard (2019). *Novacene: The Coming Age of Hyperintelligence*. Cambridge, MA: MIT Press.

Lu, C., C. Lu, R. T. Lange, et al. (2026). "Towards end-to-end automation of AI research". In: *Nature*. DOI: 10.1038/s41586-026-10265-5.

Magagna, B., I. Rosati, M. Stoica, et al. (2021). "The I-ADOPT Interoperability Framework for FAIRer data descriptions of biodiversity". In: arXiv: 2107.06547 [cs.AI].

Magesh, V., F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho (2025). "Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools". In: *Journal of Empirical Legal Studies*. DOI: 10.1111/jels.12413.

Meinhardt, C., J. N. Pava, C. Yee, and J. A. Landay (2026). *The Commercial Landscape of AI Sovereignty Offerings*. Tech. rep. Stanford HAI Policy & Society Issue Brief.

Mishra, G. (2026). "Ecology is not yet ready for AI—and why that matters". In: *Proceedings of the National Academy of Sciences*. DOI: 10.1073/pnas.2533976123.

Mitchell, M. (2026). *Jagged Intelligence: The Dangerous Unknowns at the Heart of LLMs*. The Yale Review. URL: https://yalereview.org/article/melanie-mitchell-jagged-intelligence.

Monshed, Y. (2026). *Running Ollama Locally with Podman on Fedora Linux*. Fedora Magazine. URL: https://fedoramagazine.org/running-ollama-locally-with-podman-on-fedora-linux/.

Müller, V., L. Steels, and E. Szathmáry (2026). "Evolvable AI: Threats of a new major transition in evolution". In: *Proceedings of the National Academy of Sciences*. DOI: 10.1073/pnas.2527700123.

Nagraj, V. P., K. H. Siller, T. Stewart, N. Magee, and S. D. Turner (2026). "Ten quick tips to SNIFF out sustainable and secure scientific software". In: *PLOS Computational Biology* 22.7, pp. 1–9. DOI: 10.1371/journal.pcbi.1014510.

Nature Methods Editorial (2026). "Using AI responsibly in scientific publishing". In: *Nature Methods* 23.2, p. 271. DOI: 10.1038/s41592-026-03020-1.

Neumann, J. von (1955). "Can We Survive Technology?" In: *The Neumann Compendium*. Fortune, pp. 658–673.

OpenAI (2026a). *Industrial Policy for the Intelligence Age: Ideas to Keep People First*. Tech. rep. OpenAI.

— (2026b). *OpenAI and Hugging Face partner to address security incident during model evaluation*. OpenAI. URL: https://openai.com/index/hugging-face-model-evaluation-security-incident/.

Pasche, E., J. Gobeill, L. Penev, et al. (2026). "How Biodiversity PMC can support literature search for biodiversity research". In: *World Biodiversity Forum 2026*. DOI: 10.5194/wbf2026-511.

Peppin, A., A. Reuel, S. Casper, et al. (2025). "The Reality of AI and Biorisk". In: arXiv: 2412.01946 [cs.AI].

Piskala, D. B. (2026). "From "Everything is a File" to "Files Are All You Need": How Unix Philosophy Informs the Design of Agentic AI Systems". In: arXiv: 2601.11672 [cs.SE].

Placitelli, A. P., S. Hochscheid, F. Maffucci, A. F. Rees, G. Treglia, and A. Staiano (2026). "Towards decentralized ecological monitoring: A local-first web-based AI platform for sea turtle detection". In: *Ecological Informatics*. DOI: 10.1016/j.ecoinf.2025.103569.

PomBase and ELIXIR Curation Consortium (2026). "Empowering biological knowledge-bases: advances in human-in-the-loop AI-driven literature curation". In: *Bioinformatics Advances* 6.1, vbag028. DOI: 10.1093/bioadv/vbag028.

Pudasaini, S., D. Lillis, L. Miralles-Pechuán, and M. Llorens Salvador (2026). "Why AI-Generated Text Detection Fails: Evidence from Explainable AI Beyond Benchmark Accuracy". In: arXiv: 2603.23146 [cs.CL].

Qasemi, B. (2026). *AI-Powered Cyber Attacks: Analysis and Defense Mechanisms*. Tech. rep. Independent Researcher.

Qu, A., H. Zheng, S. Y. Ong, et al. (2026). "CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery". In: arXiv: 2604.01658 [cs.AI].

Rashidi, M. (2026). "The Balkanization of Execution-Security Research for AI Coding Agents: Isolation, Access Control, and Time-of-Check-to-Time-of-Use Vulnerabilities". In: arXiv: 2607.05743 [cs.CR].

Ritchie, D. M. and K. Thompson (1974). "The UNIX Time-Sharing System". In: *Communications of the ACM* 17.7, pp. 365–375. DOI: 10.1145/361011.361061.

Salvagno, M., F. S. Taccone, and A. G. Gerli (2023). "Can artificial intelligence help for scientific writing?" In: *Critical Care* 27, p. 75. DOI: 10.1186/s13054-023-04380-2.

Shapira, N., C. Wendler, A. Yen, et al. (2026). "Agents of Chaos". In: arXiv: 2602.20021 [cs.AI].

Sigillo, L., M. Silvestri, F. Tabaro, et al. (2026). "EMBL AI Librarian: Life-Sciences Knowledge Layer for AI Agents". In: arXiv: 2607.28229 [cs.CL].

Singh, J. K., B. K. Daniel, and J. H. L. Koh (2025). "Empowering Authorship with AI: a Novel Academic Writing Technology for Authorial Voice". In: *International Journal of Artificial Intelligence in Education* 35, pp. 3406–3428. DOI: 10.1007/s40593-025-00503-8.

Slattery, P., A. K. Saeri, E. A. C. Grundy, et al. (2026). "The AI risk repository: A meta-review, database, and taxonomy of risks from artificial intelligence". In: *Patterns* 7.5. DOI: 10.1016/j.patter.2026.101517.

Smeros, P., V. Emonet, R. Wang, A.-C. Sima, and T. Mendes de Farias (2025). "SPARQL-LLM: Real-Time SPARQL Query Generation from Natural Language Questions". In: arXiv: 2512.14277 [cs.IR].

Solé, R., G. Ruffini, F. Castaldo, et al. (2026). "Large-Language Models as a Cognitive Virus". In: arXiv: 2609.03344 [physics.soc-ph].

Song, Y., Y. Song, T. Pfister, and J. Yoon (2026). "PaperOrchestra: A Multi-Agent Framework for Automated AI Research Paper Writing". In: arXiv: 2604.05018 [cs.AI].

Stanford Institute for Human-Centered Artificial Intelligence (2026a). *Artificial Intelligence Index Report 2026*. Tech. rep. Stanford University. URL: https://aiindex.stanford.edu/.

— (2026b). *Companies That Buy and Sell Your Data Are Not Following California's Strict Privacy Laws*. Stanford HAI News.

Stanford Institute for Human-Centered Artificial Intelligence (2026c). *Open-Weight Models Aren't Enough. We Need Truly Open Source AI Models for Science and Society*. Stanford HAI News. URL: https://hai.stanford.edu/news/open-weight-models-arent-enough.

— (2026d). *The Complexities of Governing Mental Health AI*. Stanford HAI News.

Takola, E. (2026). "From LSA to LLM: Evolution and limitations of topic modelling methods for biodiversity conservation". In: *Ecological Informatics* 95, p. 103725. DOI: 10.1016/j.ecoinf.2026.103725.

Theodosopoulou, D. (2022). *The Bioinformatics Landscape in Greek Environmental Omics*. Tech. rep. ELIXIR-GR Pilot Action: Marine, IMBBC, HCMR.

Turner, S. D. (2026). *AI Biosecurity Benchmarks and Real-World Risk*. Tech. rep. University of Virginia School of Data Science.

United Nations Environment Programme (2026). *EnvironmentGPT: What it is and how it works*. Tech. rep. UNEP.

Vernadsky, V. I. (1938). "The Transition From the Biosphere To the Noösphere". In: *Scientific Thought as a Planetary Phenomenon*. Translated by William Jones, 21st Century Science & Technology, Spring-Summer 2012.

Vidal, C. (2024). "What is the noosphere? Planetary superorganism, major evolutionary transition and emergence". In: *Systems Research and Behavioral Science*. DOI: 10.1002/sres.2997.

Wilson, M. L. and V. Tchantchaleishvili (2013). "The Importance of Free and Open Source Software and Open Standards in Modern Scientific Publishing". In: *Publications* 1.2, pp. 49–55. DOI: 10.3390/publications1020049.

Xu, W., E. Poussi, Q. Zhong, et al. (2026). "PantheonOS: An Evolvable Multi-Agent Framework for Automatic Genomics Discovery". In: *bioRxiv*. DOI: 10.64898/2026.02.26.707870.

Yu, W., H. Qu, S. Wang, et al. (2026). "From microbiomes to predictive ecosystems: challenges and opportunities in artificial intelligence-based approaches". In: *The Lancet Microbe*, p. 101428. DOI: 10.1016/j.lanmic.2026.101428.

Zeng, Z., X. Wang, Z. Luo, et al. (2026). "OmicClaw: executable and reproducible natural-language multi-omics analysis over the unified OmicVerse ecosystem". In: *bioRxiv*. DOI: 10.64898/2026.03.13.711464.

Zhang, C. B. C., C. Q. Knight, N. Kruus, et al. (2026). "LLM Novice Uplift on Dual-Use, In Silico Biology Tasks". In: arXiv: 2602.23329 [cs.AI].

Zhang, W., Y. Li, Y. Bei, et al. (2025). "From Web Search towards Agentic Deep Research: Incentivizing Search with Reasoning Agents". In: arXiv: 2506.18959 [cs.IR].

Zhao, Z., Y. Wang, T. Stuart, M. De Vaan, P. Ginsparg, and Y. Yin (2026). "LLM hallucinations in the wild: Large-scale evidence from non-existent citations". In: arXiv: 2605.07723 [cs.DL].